MÓDULO 1.0: INFRAESTRUCTURA "LOW-COST / HIGH-PERFORMANCE" (EL HIERRO)

Este módulo arregla tu arquitectura Hetzner para que no colapse.

- **1.1. Configuración del Núcleo (Hetzner Bare Metal):**
  - 1.1.1. Selección de Servidor AX102 vs. Subasta de GPU (Análisis Coste/Rendimiento).
  - 1.1.2. Virtualización Ligera: Implementación de **Proxmox** o **Harvester** para gestión de contenedores sin el "impuesto" de Kubernetes.
  - 1.1.3. Red de Almacenamiento: Configuración de ZFS con caché NVMe para velocidad de lectura/escritura en bases de datos masivas.
- **1.2. El Stack de IA "Soberano" (Sin pagar a OpenAI):**
  - 1.2.1. Despliegue de **vLLM** (Motor de inferencia de alta velocidad) en Docker.
  - 1.2.2. Selección y Cuantización de Modelos: Implementación de **Mistral-Nemo-12B** (mejor ratio inteligencia/velocidad que Llama 3 para este caso) cuantizado a AWQ 4-bit.
  - 1.2.3. Pipeline de "Ingeniería Inversa" de APIs: Configuración de **Mitmproxy** para interceptar tráfico de Apps Móviles (Mobile.de/Autoscout) y ahorrar un 90% de ancho de banda.
- **1.3. Red de Anonimato (Evasión de Bloqueos):**
  - 1.3.1. Configuración de **Glutetun** (VPN Client) con rotación de IPs residenciales baratas (ej: Webshare o IPRoyal).
  - 1.3.2. Desarrollo de "Fingerprint Spoofing": Scripts para engañar a Cloudflare/Datadome simulando ser un usuario real de Chrome en Windows.

MÓDULO 2.0: ESTRATEGIA DE SCRAPING TOTAL ("DEBAJO DE LAS PIEDRAS")

Aquí entra tu petición de buscar talleres en la montaña y redes sociales.

- **2.1. Capa 1: Los Gigantes (Portales Tier 1):**
  - 2.1.1. Desarrollo de conectores API inversos para Mobile.de, AutoScout24, Leboncoin, Coches.net.
  - 2.1.2. Estrategia de "Diff Scraping": Solo descargar lo que ha cambiado para ahorrar recursos.
- **2.2. Capa 2: Deep Local & Social (El Océano Azul):**
  - 2.2.1. **Agente "Google Maps crawler":** Script para barrer coordenadas geográficas buscando negocios categorizados como "Taller", "Concesionario", "Desguace".
  - 2.2.2. **Agente "Social Hunter":** Scraping de Facebook Marketplace e Instagram (búsqueda por hashtags geolocalizados #sevendecoche #madrid).
  - 2.2.3. Extracción de WhatsApp/Teléfono: OCR (Lectura de imágenes) para sacar teléfonos que la gente pone en las fotos de los coches para evitar bots.
- **2.3. Normalización Forense (El Esperanto del Coche):**
  - 2.3.1. Mapeo de "Diccionario Maestro": Unificar "Techo solar", "Schiebedach" y "Toit ouvrant" en un solo ID.

    - 2.3.2. Detección de IVA: Algoritmo regex para identificar si un coche es REBU, IVA Deducible o Neto.

FASE 2: INTELIGENCIA Y PRODUCTO ("EL CEREBRO")

Objetivo: Convertir datos brutos en dinero (Arbitraje y Valoración).

MÓDULO 3.0: MOTOR DE ARBITRAJE Y PRICING (EL INDICATA-KILLER)

- **3.1. Algoritmo de Valoración en Tiempo Real:**
    - 3.1.1. Cálculo de "Precio Justo": Regresión lineal basada en Marca, Modelo, Año, Km y Potencia.
    - 3.1.2. Ajuste por Liquidez: Penalización del precio si el modelo tarda más de 60 días en venderse en esa región.
- **3.2. La Matriz de Arbitraje Fiscal (El "Santo Grial"):**
    - 3.2.1. Calculadora DE->ES: Coste coche + Transporte + Impuesto Matriculación (CO_2) + Gestoría vs. Precio Mercado España.
    - 3.2.2. Calculadora DE->FR: Cálculo del "Malus Ecologique" francés (crítico para no perder dinero).
- **3.3. Scoring de Oportunidad (El Semáforo):**
    - 3.3.1. Definición de KPIs: Margen Neto %, Días para Vender, Riesgo de Fraude.
    - 3.3.2. Sistema de Alertas "Sniper": Notificación push en <5 minutos cuando aparece una "Gema" (Margen >20%).

MÓDULO 4.0: EL HISTORIAL UNIFICADO (LA VERDAD GRATIS)

Tu petición de historial gratuito/barato.

- **4.1. Estrategia OSINT (Inteligencia de Fuentes Abiertas):**
    - 4.1.1. Rastreo de VIN en Google Imágenes y Archivos de Subastas: Detectar si el coche tuvo un golpe en el pasado buscando fotos antiguas cacheadas.
    - 4.1.2. Cross-Check de Kilometraje: Si el coche se anunció hace 2 años con 150k km y hoy tiene 90k km -> **ALERTA DE FRAUDE GRATIS**.
- **4.2. Integración Oficial "Low Cost":**
    - 4.2.1. Conexión directa DGT (España) / SIV (Francia) / RDW (Holanda - Gratis).
    - 4.2.2. Modelo de negocio "Freemium": Datos básicos (Robo, Embargo) gratis; Datos detallados (Dueños, ITV completas) a coste de tasa administrativa.

FASE 3: ECOSISTEMA B2B Y SERVICIOS ("EL SISTEMA OPERATIVO")

Objetivo: Que el dealer viva dentro de CARDEX.

MÓDULO 5.0: EL CRM "INVISIBLE" Y GESTIÓN DE STOCK

Para el autónomo con 5 coches que no quiere líos.

- **5.1. Onboarding sin Fricción:**
  - 5.1.1. Feature "Importar desde Wallapop": El dealer pega el link de sus coches y nosotros le creamos el inventario en CARDEX automáticamente.
  - 5.1.2. Generación de Fichas PDF con QR para poner en el coche físico.
- **5.2. Gestión Documental Automatizada:**
  - 5.2.1. Contratos de Compraventa Auto-rellenables (DNI escaneado con IA).
  - 5.2.2. Libro de Policía Digital (Obligatorio para compraventas).

MÓDULO 6.0: AGENCIA DE MARKETING AUTOMATIZADA

Tu idea de venderles servicios de marketing.

- **6.1. Auditoría Automática de Dealers:**
  - 6.1.1. El scraper analiza al dealer: ¿Sus fotos son malas? ¿No tiene web? ¿Sus descripciones son cortas?
  - 6.1.2. Generación de "Informe de Mejora": Se le envía un PDF automático diciéndole cuánto dinero pierde por su mal marketing.
- **6.2. Soluciones "Un Clic":**
  - 6.2.1. **Multiposting:** "Sube el coche a Cardex y nosotros lo publicamos en Coches.net, Wallapop y Facebook por ti".
  - 6.2.2. **Web Instantánea:** "Tu propia web de concesionario generada en 30 segundos con tu stock actual".

MÓDULO 7.0: MARKETPLACE DE SERVICIOS FÍSICOS

- **7.1. Inspecciones In-Situ ("Uber de Mecánicos"):**
  - 7.1.1. Red de peritos freelance: Geolocalización de mecánicos.
  - 7.1.2. Protocolo de revisión estandarizado: App móvil para que el mecánico haga el checklist y suba fotos en tiempo real.
- **7.2. Logística y Transporte:**
  - 7.2.1. Agregador de camiones: Subasta inversa de huecos en camiones porta-coches (tipo Clicktrans).

FASE 4: ESTRATEGIA CORPORATIVA Y LEGAL

- **MÓDULO 8.0:** Modelo de Negocio, Unit Economics y Pricing.
- **MÓDULO 9.0:** Legal, GDPR y Protección de Datos (Blindaje ante demandas de portales).
- **MÓDULO 10.0:** Plan de Contingencia y "War Room" (Qué hacer si Mobile.de nos bloquea mañana).

1.1.1. INFRAESTRUCTURA DE CÓMPUTO DE ALTO RENDIMIENTO (HPC) - ESPECIFICACIÓN TÉCNICA FINAL

ESTADO: APROBADO PARA EJECUCIÓN **OBJETIVO:** PROCESAMIENTO DE 12 MILLONES DE ACTIVOS + HISTÓRICO **FILOSOFÍA:** SOBERANÍA DEL DATO Y RENDIMIENTO BRUTO (BARE METAL)

1. DECISIÓN DE HARDWARE: EL NÚCLEO "AX102"

Para gobernar un sistema que debe ingestar, normalizar y servir datos de 12 millones de vehículos en tiempo real, desechamos servidores genéricos. Estandarizamos la flota sobre una única unidad de combate: el **Hetzner AX102**.

No es una elección aleatoria. Es la única configuración matemática que ofrece el rendimiento de un mainframe bancario a precio de consumo.

LA ESPECIFICACIÓN DE LA UNIDAD (THE UNIT):

- **CPU: AMD Ryzen™ 9 7950X3D**.
    - *Por qué:* La tecnología 3D V-Cache de este procesador es crítica. Para bases de datos masivas (ClickHouse/Postgres) y lógica de scraping compleja, la caché L3 masiva (128MB) reduce la latencia de memoria drásticamente. Supera a procesadores Intel Xeon de coste triple en tareas de mono-hilo y bases de datos transaccionales.
    - *Uso:* Orquestación de Agentes, Motor de Base de Datos, API Server.
- **RAM: 128 GB DDR5 ECC**.
    - *Por qué:* DDR5 ofrece un ancho de banda un 50% superior a DDR4. La corrección de errores (ECC) es **obligatoria** a nivel institucional. No podemos permitir que un bit rotado por radiación cósmica corrompa el precio de un activo financiero o un contrato en el CRM.
- **ALMACENAMIENTO: 2x 3.84 TB NVMe SSD Datacenter Edition (Gen4)**.
    - *Por qué:* Necesitamos IOPS (Operaciones de Entrada/Salida por segundo) masivas para escribir los logs de 12 millones de coches. Los discos "Datacenter Edition" tienen una resistencia a la escritura (TBW) muy superior a los discos de consumo.
    - *Configuración:* **RAID 1 (Espejo)** por software (ZFS). Si un disco muere, el sistema no se detiene. Cero tiempo de inactividad.
- **RED: 1 Gbit/s ancho de banda garantizado**.
    - *Por qué:* El scraping masivo consume ancho de banda constante, no picos.

---

2. ARQUITECTURA DE VIRTUALIZACIÓN (EL SISTEMA OPERATIVO)

No instalaremos Linux "a pelo". Implementaremos un hipervisor de grado empresarial para segmentar la carga y aislar fallos.

DECISIÓN TÉCNICA: PROXMOX VE (VIRTUAL ENVIRONMENT)

- **Fundamento:** Proxmox nos permite dividir el AX102 en "contenedores ligeros" (LXC) que comparten el kernel del host. Esto elimina el "impuesto" de rendimiento de las máquinas virtuales tradicionales (VMs) manteniendo el aislamiento de seguridad.
- **Sistema de Archivos: ZFS (Zettabyte File System).**
    - *Compresión:* Activaremos **LZ4** en tiempo real. Esto convertirá nuestros 7.6 TB de disco físico en aproximadamente 12-15 TB de almacenamiento lógico efectivo para texto (JSON/HTML de coches), ahorrando miles de euros en discos duros extra.
    - *Integridad:* ZFS realiza checksums de cada bloque de datos. Si un dato se corrompe en el disco, ZFS lo detecta y lo repara automáticamente usando la copia del espejo RAID 1. **El dato es sagrado.**

---

3. TOPOLOGÍA DE DESPLIEGUE (CÓMO SE USA)

Un solo servidor no soporta 12 millones de coches con IA. Desplegaremos un **Cluster Inicial de 3 Nodos** idénticos (AX102), segregando funciones para evitar el "ruido de vecinos".

NODO 01: "EL CEREBRO" (CORE & DATA)

- **Función:** Exclusivo para Bases de Datos y API.
- **Carga:**
    - *PostgreSQL (TimescaleDB):* Usuarios, Mandatos, CRM.
    - *ClickHouse:* El almacén de Big Data para los 12M de coches y sus históricos de precios.
    - *Redis:* Caché de alta velocidad para la UI.
- **Justificación:** Las bases de datos necesitan toda la RAM y IOPS del disco. No pueden competir con scrapers.

NODO 02: "LA HIDRA" (INGESTA & SCRAPING)

- **Función:** Lanzar y gestionar los miles de micro-browsers y peticiones HTTP.
- **Carga:**
    - *Crawlee / Scrapy Cluster:* Los agentes que visitan las webs.
    - *Mitmproxy:* La intercepción de tráfico de Apps móviles.
- **Justificación:** El scraping consume mucha CPU (parseo de HTML) y abre miles de conexiones de red. Si este nodo se satura o es baneado, el "Cerebro" (Nodo 01) sigue sirviendo datos a los clientes sin interrupción.

NODO 03: "EL ANALISTA" (IA LOCAL & PROCESAMIENTO)

- **Función:** Inferencia de IA y Normalización pesada.
- **Carga:**
  - *vLLM / Ollama:* Modelos de lenguaje pequeños (Mistral/Qwen) ejecutándose en CPU para limpieza de texto.
  - *Normalizadores:* Scripts que convierten "Techo solar panorámico" -> `sunroof: true`.
- **Nota Crítica:** Al no tener GPU dedicada en el AX102, este nodo se usará para tareas de IA optimizadas para CPU (cuantización extrema) y como "buffer" para enviar tareas pesadas a APIs externas si fuera necesario.

---

4. PROTOCOLO DE SEGURIDAD (HARDENING)

- **Firewall Perimetral:** Hetzner Cloud Firewall configurado ANTES del servidor. Solo puertos 80/443 (Web) y puerto de gestión (VPN) abiertos.
- **Red Privada (vSwitch):** Los 3 nodos se conectan entre sí por una red privada virtual (VLAN) de 10Gbps. El tráfico de base de datos entre el Nodo 2 (Scraper) y el Nodo 1 (Base de Datos) **nunca** toca internet público. Es invisible y ultrarrápido.
- **Encriptación:** Discos cifrados con LUKS. Si alguien roba el físico del servidor en el datacenter, no saca ni un solo dato de nuestros clientes.

EL CLUSTER DE 3 NODOS (AX102)

- **Coste Unitario (1 Servidor):** ~109,00 € / mes.
- **Coste Total Cluster (3 Servidores): 327,00 € / mes.**
  - *(Nota: Existe un coste de alta "Setup Fee" de pago único aprox. de 117€).*

LA COMPARATIVA DE VALOR (EL GOLPE DE REALIDAD)

- **Tu Cluster (327 €/mes):** Te da **48 Núcleos** (frecuencia 5.7GHz), **384 GB RAM DDR5** y **23 TB** de almacenamiento NVMe Gen4.
- **Equivalente en AWS (3x r6i.4xlarge + EBS):** Te costaría aproximadamente **2.800 € / mes** + costes de transferencia de datos

1.1.2. ARQUITECTURA DE VIRTUALIZACIÓN: EL SISTEMA INMUNOLÓGICO

DECISIÓN TÉCNICA ÚNICA: PROXMOX VE (VIRTUAL ENVIRONMENT). No VMware (licencias caras). No Kubernetes Bare Metal (demasiada complejidad de gestión para 3 nodos).

1. LA ESTRATEGIA DE "CONTENCIÓN HÍBRIDA"

No trataremos todas las cargas de trabajo igual. Usaremos dos tecnologías distintas dentro de Proxmox para exprimir el hardware:

A. Para "El Cerebro" (Bases de Datos & Core): LXC (Linux Containers)

- **Qué es:** Contenedores de sistema. Comparten el Kernel del host (AX102).
- **Por qué: Rendimiento Nativo (Zero-Overhead).**
- **La Ventaja Injusta:** PostgreSQL y ClickHouse correrán dentro de contenedores LXC. A efectos prácticos, tienen acceso directo a la CPU y la RAM sin la capa de traducción de una Máquina Virtual.
    - *Resultado:* 99% de rendimiento nativo vs el 85-90% de una VM tradicional. En 12 millones de registros, ese 10% es la diferencia entre respuesta instantánea o lag.

B. Para "La Hidra" (Scrapers & IA): KVM (Kernel-based Virtual Machine)

- **Qué es:** Virtualización completa de hardware.
- **Por qué: Aislamiento Total.**
- **La Ventaja Injusta:** Los scrapers son procesos sucios, inestables y peligrosos (pueden bloquearse, consumir toda la RAM o ser atacados). Si encerramos a los scrapers en una VM KVM y explotan, el resto del servidor ni se entera. "El Cerebro" sigue operando.

---

2. EL SISTEMA DE ARCHIVOS: ZFS TUNING (EL SECRETO DEL RENDIMIENTO)

El hardware es potente, pero si lo configuras mal (EXT4 estándar), lo desperdicias. Usaremos **ZFS** con una configuración agresiva para NVMe.

CONFIGURACIÓN "GOD-MODE" DEL STORAGE:

1. **Compresión LZ4 en Tiempo Real:**
    - *Orden:* `zfs set compression=lz4 rpool`
    - *Efecto:* Los datos de texto (HTML, JSON de coches) se comprimen *antes* de escribirse al disco.
    - *Impacto:* Convertimos tus 7.6 TB físicos en **~12-15 TB lógicos**. Además, al escribir menos datos físicos, **alargamos la vida útil de los discos NVMe** (que sufren con la escritura intensiva).
2. **ARC (Adaptive Replacement Cache) Inteligente:**
    - ZFS usa la RAM libre como caché de disco ultrarrápida.

    - *Ajuste:* Asignaremos 64GB de RAM (de los 128GB) exclusivamente para caché de lectura. Los datos más consultados (ej: "Precios de BMW Serie 3") vivirán en la RAM. Respuesta en **nanosegundos**, no milisegundos.
3. **Snapshots Atómicos:**
    - Antes de cualquier despliegue de código o actualización de base de datos, el sistema toma una "foto" (Snapshot) del disco. Tarda 1 segundo.
    - *Si algo sale mal:* `zfs rollback`. En 1 segundo volvemos al estado anterior. **Inmunidad contra errores humanos.**

---

3. LA RED INTERNA (VIRTUAL SWITCHING)

No dejaremos que el tráfico interno (Scraper enviando datos a la Base de Datos) salga a la tarjeta de red física y vuelva a entrar. Eso añade latencia.

- **Linux Bridge (vmbr0):** Configuraremos un puente virtual en memoria.
- **Efecto:** La comunicación entre el contenedor de PostgreSQL y la VM de Scraping ocurre a la velocidad del bus de memoria del procesador (**>50 Gbit/s** virtuales).
- **Seguridad:** Este tráfico es invisible desde fuera. Ni siquiera el firewall perimetral lo ve.

1.2. EL STACK DE IA "SOBERANO" E HÍBRIDO (SIN PAGAR EL "IMPUESTO OPENAI")

OBJETIVO: Procesar millones de campos de texto (descripciones de coches, equipamiento) a coste marginal cero en local, y externalizar el razonamiento complejo a proveedores ultra-baratos.

1.2.1. MOTOR DE INFERENCIA LOCAL (CPU-FIRST)

Olvídate de PyTorch crudo. Es lento y pesado. Para correr modelos en CPU (Ryzen) a la velocidad del rayo, necesitamos metal puro.

DECISIÓN TÉCNICA: OLLAMA (DOCKERIZADO) + AVX-512

- **El Núcleo:** Usaremos **Ollama** como servidor de inferencia. ¿Por qué? Porque envuelve `llama.cpp`, la librería más eficiente del mundo para correr modelos en CPU usando instrucciones vectoriales (AVX-512) que tu Ryzen 9 soporta nativamente.
- **La Configuración "God-Mode":**
    - No lo instalaremos como app de escritorio. Lo desplegaremos en el **Nodo 03 (El Analista)** como un contenedor Docker optimizado.

- **Variables de Entorno Críticas:**
  - `OLLAMA_NUM_PARALLEL=8`: Permitiremos procesar 8 descripciones de coches simultáneamente. El Ryzen tiene 16 núcleos físicos (32 hilos). Dedicamos la mitad a la IA, el resto al sistema.
  - `OLLAMA_MAX_LOADED_MODELS=2`: Mantendremos solo 2 modelos pequeños en RAM para evitar el *swapping* (que mataría el rendimiento).

1.2.2. SELECCIÓN Y CUANTIZACIÓN DE MODELOS (LA ARTILLERÍA)

No usaremos modelos gigantes (70B/405B) en local. Ahogarían el servidor. Usaremos **SLMs (Small Language Models)** especializados.

EL ARSENAL LOCAL (COSTO 0€):

1. **El Extractor Estructural: `Qwen-2.5-7B-Instruct` (Quant: Q4_K_M)**
   - *Misión:* Extraer JSON limpio de texto sucio.
   - *Ejemplo:* Convierte "lleva techo y asientos calefactables" -> `{"sunroof": true, "heated_seats": true}`.
   - *Por qué:* Qwen 2.5 bate a Llama 3 en adherencia a formatos JSON y matemáticas. En 4-bits (Q4) ocupa solo ~5GB de RAM y vuela en tu CPU.
2. **El Clasificador Rápido: `Phi-3.5-mini` (3.8B) (Quant: Q5_K_M)**
   - *Misión:* Triaje rápido. ¿Es este anuncio un coche o una estafa? ¿Es un particular o un profesional?
   - *Por qué:* Es minúsculo. Responde en milisegundos. Es nuestro "portero de discoteca".

EL ARSENAL EN LA NUBE (COSTO INFRA-MARGINAL): *Para cuando el local no basta.*

3. **El Cerebro Pesado: API de `DeepSeek-V3`**
   - *Misión:* Análisis de sentimiento ("¿El vendedor está desesperado?"), traducción compleja y redacción de emails de contacto.
   - *Coste:* ~$0.14 por millón de tokens de entrada. Es 60 veces más barato que GPT-4o.
   - *Router Inteligente:* Implementaremos un script en Python (`AiRouter.py`) que decide: *"¿Es tarea fácil? -> Local (Gratis). ¿Es tarea difícil? -> DeepSeek (Céntimos)."*

1.2.3. PIPELINE DE "INGENIERÍA INVERSA" DE APIs (EL ARMA SECRETA)

Este punto está ligado a la ingesta, pero se define aquí porque requiere software especializado.

No vamos a scrapear HTML visual (lento, pesado, lleno de publicidad). Vamos a atacar las **APIs Privadas** de las Apps Móviles (Android/iOS) de Mobile.de, AutoScout24, etc.

EL PROTOCOLO DE INTERCEPTACIÓN:

1. **La Herramienta: `Mitmproxy` (Man-In-The-Middle Proxy)**
    - Desplegaremos una instancia de Mitmproxy en el **Nodo 02**.
    - Todo el tráfico de nuestros agentes pasará por aquí. Nos permite ver, grabar y *replay* (repetir) las peticiones exactas que hace la App oficial.
2. **La Llave Maestra: `Frida` + `Objection` (SSL Unpinning)**
    - Las Apps modernas usan "SSL Pinning" para rechazar certificados que no sean los suyos.
    - *Acción:* Usaremos scripts de **Frida** (kit de instrumentación dinámica) para inyectar código en tiempo de ejecución en las APKs de los portales y desactivar la validación SSL.
    - *Resultado:* Tendremos acceso al tráfico JSON puro y limpio.
    - *Ventaja:* Una respuesta JSON pesa 5KB. La página web equivalente pesa 2MB. **Reducimos el ancho de banda y el uso de CPU un 99%.**

1.3. RED DE ANONIMATO Y EVASIÓN ACTIVA ("EL MANTO")

OBJETIVO: Que los servidores de Mobile.de, AutoScout24 y Leboncoin crean que somos 12 millones de usuarios humanos distintos con iPhones y PCs domésticos, y no tres servidores bestiales en un datacenter alemán.

LA CRUDA REALIDAD: Las IPs de Hetzner (y AWS/Google) tienen un "Trust Score" de 0. Son radiactivas. Si sales a internet "a pelo" desde el AX102, estás muerto antes de empezar.

1.3.1. ENRUTAMIENTO INTELIGENTE DE TRÁFICO (EL "PROXY ROUTER")

No vamos a comprar una suscripción de proxies y pegarla en el código. Eso es basura inmanejable. Construiremos un **Gateway de Salida Unificado**.

DECISIÓN TÉCNICA: GOST (GO SIMPLE TUNNEL) + HAPROXY Desplegaremos en el **Nodo 02 (La Hidra)** un contenedor que actuará como la única puerta de salida a internet para los scrapers.

- **La Lógica de Combate (Routing Rules):**
    - El Router analiza el dominio destino antes de enviar el paquete.
    - **Si el destino es Tier 1 (Mobile.de):** Enruta por el túnel de **Proxies Residenciales Rotativos de Alta Calidad** (Bright Data / Oxylabs). Coste alto, tasa de éxito 99.9%. Solo para el HTML crítico.

- **Si el destino es Tier 2 (Web de concesionario pequeño):** Enruta por **Proxies de Datacenter Baratos** (Webshare). Coste ridículo, velocidad máxima.
- **Si el destino es Imágenes/Assets:** Enruta por la **IP de Hetzner** (Directa). No gastamos ancho de banda caro en descargar JPEGs. A los servidores no les importa quién descarga la foto, solo quién pide el HTML. **Ahorro del 80% en factura de proxies.**

1.3.2. GESTIÓN DE IDENTIDAD DE RED (TLS FINGERPRINTING)

Aquí es donde los "seniors" fallan. Los WAFs modernos (Cloudflare) no miran tu IP, miran tu "saludo" SSL (TLS Handshake).

- **El Problema:** Python `requests` o `urllib` tienen una firma TLS (JA3 Signature) que grita "SOY UN SCRIPT". Bloqueo inmediato aunque uses la mejor IP del mundo.
- **La Solución "God-Mode":** Implementación de **CURL-IMPERSONATE** y librerías **TLS-CLIENT**.
  - Modificaremos nuestros scrapers para que, a nivel de paquete TCP/IP, el handshake criptográfico sea **bit a bit idéntico** al de Chrome 120 o Safari iOS 17.
  - Rotaremos no solo la IP, sino la "personalidad criptográfica" en cada petición.

1.3.3. EL BÚNKER DE NAVEGACIÓN (BROWSER SPOOFING)

Para los sitios que requieren JavaScript (Browser-Use), no basta con un navegador headless estándar. Son detectables por el tamaño de la ventana, la falta de tarjeta gráfica o las fuentes instaladas.

PROTOCOLOS DE MIMETIZACIÓN:

1. **Canvas & WebGL Noise:** Inyectaremos "ruido" aleatorio en el renderizado gráfico. Si el servidor nos pide "dibuja un cubo", nuestro cubo será microscópicamente único, impidiendo que nos rastreen por la huella de hardware del servidor.
2. **Mouse Movements Humanizados (Curvas de Bezier):** Los bots mueven el ratón en línea recta (A -> B). Nuestros agentes moverán el ratón con curvas, aceleración y "micro-dudas" (overshoot) antes de hacer clic.
3. **Font & Plugin Enumeration:** El contenedor Docker inyectará listas de fuentes y plugins falsos para que parezca un Windows/Mac normal, no un Linux de servidor pelado.

1.3.4. SOLUCIÓN DE CAPTCHAS AUTOMATIZADA (EL ROMPE-MUROS)

A pesar de todo, a veces saltará un CAPTCHA. No podemos detener la máquina.

- **Integración de CapSolver / 2Captcha:** API directa en el Proxy Router.
- **Flujo:** Si el HTML de respuesta contiene el patrón de "Captcha", el Router congela la petición, envía el desafío a la granja de resolución, recibe el token de validación,

lo inyecta y devuelve la página limpia al scraper. El scraper ni se entera de que hubo un captcha. Transparencia total.

MÓDULO 2.1.1: PROTOCOLO "API GHOST" (INGENIERÍA INVERSA DE APLICACIONES NATIVAS)

OBJETIVO: Extraer el flujo de datos JSON puro de las aplicaciones móviles (Mobile.de, AutoScout24, Leboncoin, etc.) evitando la capa de presentación HTML y reduciendo el consumo de ancho de banda en un 90%.

EL PROBLEMA: Estas aplicaciones están blindadas con **SSL Pinning** (rechazan certificados proxy), **Root Detection** (se cierran si detectan un entorno modificado) y **Firmas Criptográficas** (headers HMAC) generadas por librerías nativas (`.so`) para validar que la petición viene de un móvil real.

LA SOLUCIÓN: Construir una **Granja de Desencriptación Híbrida** en el Nodo 02 (La Hidra).

A. INFRAESTRUCTURA DEL LABORATORIO DE INTERCEPTACIÓN

No scrapeamos desde un móvil físico en mano. Virtualizamos la ejecución.

1. **El Entorno de Ejecución (Containerized Android):**
    - **Tecnología: ReDroid (Remote Android in Docker)**.
    - **Implementación:** Desplegamos contenedores Docker que ejecutan una versión de Android (AOSP) ligera y optimizada en el servidor Hetzner.
    - **Configuración del Kernel:** El kernel del host (Proxmox/Linux) necesita módulos específicos (`binder`, `ashmem`) para soportar Android.
    - **Ventaja:** Podemos levantar 50 instancias de Android virtuales consumiendo mucha menos RAM que emuladores tradicionales (AVD).
2. **El Camuflaje del Sistema (Root Hiding):**
    - Las Apps Tier 1 detectan si el dispositivo está rooteado.
    - **Herramienta: KernelSU** o **Magisk** con **Zygisk** (Zygote Injection).
    - **Módulo "Shamiko":** Oculta el root a las aplicaciones objetivo. Para Mobile.de, nuestro contenedor Android parece un Samsung Galaxy S23 no modificado.

B. RUPTURA DEL CIFRADO (SSL UNPINNING)

Necesitamos leer el tráfico HTTPS. Para ello, debemos inyectar nuestro certificado CA en el almacén de confianza del sistema Android y forzar a la App a aceptarlo.

1. **Inyección Dinámica:**
    - **Herramienta: Frida Server** corriendo dentro del contenedor Android.
    - **Script de Ataque:** Usaremos un script de JavaScript inyectado via Frida (basado en `universal-android-ssl-pinning-bypass-2`) que engancha (hooks) las llamadas a las librerías de red nativas (`okhttp3`, `trustmanager`, `conscrypt`).
    - **Acción:** Cuando la App intenta verificar el certificado del servidor, nuestro script intercepta la función `checkServerTrusted` y devuelve siempre `true`.
2. **El Proxy Intermediario (Mitmproxy):**
    - El tráfico desencriptado fluye desde el contenedor Android hacia una instancia de **Mitmproxy** en modo transparente.
    - **Función:** Grabar los flujos (flows) y exportar los esquemas JSON exactos.

C. ARMAMENTIZACIÓN (DE APP A PYTHON)

No queremos ejecutar la App de Android para *cada* búsqueda (es lento). Queremos replicar la petición en Python. Aquí está la clave maestra: **La Firma Digital (Request Signing).**

Las Apps mandan un header tipo `X-Signature: a1b2c3...` calculado con un algoritmo secreto dentro de la App. Si no lo tienes, el servidor te rechaza.

ESTRATEGIA 1: EXTRACCIÓN DE LA LÓGICA (REVERSING PURO)

- Desamblamos el APK usando **JADX** o **Ghidra**.
- Buscamos la función nativa (C++) que genera la firma.
- **Si es replicable:** Reescribimos el algoritmo en Python/C++ y lo integramos en el scraper. Velocidad máxima. Coste CPU mínimo.

ESTRATEGIA 2: RPC (REMOTE PROCEDURE CALL) - "EL ORÁCULO"

- *Caso:* El algoritmo de firma es demasiado complejo u ofuscado (ej: usa virtualización de código como Arxan).
- *Solución:* Mantenemos una instancia de la App corriendo en el contenedor ReDroid.
- *Implementación:*
    1. Inyectamos un script Frida que expone un servidor web local (`RPC Server`) dentro del Android.
    2. Nuestro scraper Python (externo) envía los datos de la búsqueda (ej: "BMW X5") a este servidor RPC.
    3. El script Frida llama a la función interna de la App `generateSignature("BMW X5")` y nos devuelve el token válido.
    4. El scraper Python usa ese token para hacer la petición a la API de Mobile.de a velocidad luz, sin usar la interfaz gráfica de la App.

D. GESTIÓN DE IDENTIDAD DE DISPOSITIVO (ROTACIÓN)

Para evitar que baneen nuestro "Móvil Virtual", este debe mutar.

1. **Device ID Changer:** Script que modifica el `build.prop` y los identificadores únicos (`ANDROID_ID`, `IMEI`) antes de cada reinicio del contenedor.
2. **Huella de Red:** El tráfico saliente de ReDroid se enruta por el **Proxy Router** (definido en 1.3) para que la IP coincida con la geolocalización simulada del dispositivo.

MÓDULO 2.1.2: ESTRATEGIA DE "DIFF SCRAPING" (SINCRONIZACIÓN DELTA DE ALTA EFICIENCIA)

OBJETIVO: Mantener un espejo en tiempo real de 12 Millones de activos descargando *únicamente* los bytes que han cambiado (nuevos ingresos, cambios de precio, ventas).

EL PRINCIPIO MATEMÁTICO: El mercado de VO tiene una tasa de rotación diaria del ~3-5%. Si descargas el 100% cada día, estás desperdiciando el 95% de tu presupuesto.

A. EL MOTOR DE DESCUBRIMIENTO (LA QUITANIEVES)

Cómo detectamos lo nuevo sin leer lo viejo.

La mayoría de APIs móviles permiten ordenar por `sort=creation_time_desc` (Más recientes). Usaremos esto para crear un **Bucle de Ingesta Inverso**.

1. **El "Cursor de Verdad":**
   - En nuestra base de datos (Redis/Postgres), mantenemos el `last_seen_id` (el ID del anuncio más reciente capturado con éxito) para cada combinación de Marca/Modelo/País.
   - *Ejemplo:* Para "BMW Serie 3 en Alemania", el último ID visto es `38492011`.
2. **El Algoritmo de Barrido (The Sweep):**
   - El scraper pide la **Página 1** de la API (los 20 coches más nuevos).
   - Itera sobre los resultados:
     - ¿ID `38492055`? -> Nuevo. **INGESTAR.**
     - ¿ID `38492011`? -> Conocido. **STOP.**
   - **Condición de Parada:** En el momento en que encontramos un ID que ya existe en nuestra base de datos, el scraper detiene la ejecución para esa búsqueda.
   - **Ventaja:** Si han entrado 5 coches nuevos, solo hacemos 1 petición a la API. Eficiencia máxima.
3. **Gestión de "Bumps" (Falsos Nuevos):**
   - *El Truco del Dealer:* Muchos vendedores borran y resuben el anuncio para salir arriba.

- *Contramedida:* Hashing Perceptual. Al ingestar un "nuevo" anuncio, generamos un hash de (Título + Precio + Km + Hash de 1ª Foto). Si coincide con un coche "vendido" hace <24h, lo marcamos como **RE-LISTADO**, no como nuevo. Esto es vital para no distorsionar las métricas de stock.

B. EL MONITOR DE VIDA (HEARTBEAT PROTOCOL)

Cómo detectamos lo vendido sin volver a scrapear.

Saber cuándo se vende un coche es tan importante como saber cuándo entra. Es lo que define la **Liquidez**. Pero no podemos visitar 12 millones de fichas cada hora.

1. **Estrategia de Sondeo Probabilístico (TTL Dinámico):**
   - No comprobamos todos los coches con la misma frecuencia. Asignamos una "Frecuencia de Pulso" basada en la probabilidad de venta.
   - **Coches "Hot" (Gemas):** Precio bajo, alta demanda -> Check cada **1 hora**.
   - **Coches "Zombie" (Caros, 90+ días en stock):** Check cada **3 días**.
   - **Coches Nuevos (<24h):** Check cada **4 horas**.
2. **Técnica de "Ping Ligero" (API Endpoint Optimization):**
   - No pedimos el JSON completo del detalle del coche (pesado).
   - Buscamos endpoints ligeros en la API, tipo `GET /api/v2/status?ids=[id1,id2,id3]`.
   - Si la API no tiene endpoint de estado masivo, usamos peticiones `HEAD` (solo cabeceras HTTP) o buscamos el ID en el listado de búsqueda del vendedor.
   - **Detección de Muerte:**
     - Código 404 / 410 -> **VENDIDO / ELIMINADO.**
     - Código 200 -> **ACTIVO.**
     - Redirección (301) a "Coches similares" -> **VENDIDO.** (Ojo con esto, muchos parsers fallan aquí creyendo que sigue vivo).

C. GESTIÓN DEL CAMBIO (VERSIONADO DE PRECIOS)

No sobrescribimos datos. **CARDEX tiene memoria infinita.**

1. **Arquitectura de Eventos (Event Sourcing Light):**
   - La tabla `vehicles` tiene el estado actual.
   - La tabla `price_history` (en ClickHouse/TimescaleDB) guarda cada cambio.
   - *Estructura:* `{vehicle_id, timestamp, old_price, new_price, delta_percent}`.
2. **El "Delta Trigger":**
   - Cuando el *Monitor de Vida* detecta que un coche sigue vivo, compara el precio actual con el almacenado.
   - ¿Diferente? -> **INSERCIÓN EN HISTÓRICO.**

- ¿Igual? -> Actualizamos solo el campo `last_seen_at`.
- **Inteligencia Derivada:** Esto alimenta el **SDI (Seller Desperation Index)**. Si detectamos 3 bajadas de precio en 48 horas, se dispara una alerta de "Vendedor en Pánico".

D. INFRAESTRUCTURA DE DATOS PARA DIFF (EL BACKEND)

Para que esto funcione con 12 millones de activos, el almacenamiento debe ser híbrido.

1. **Redis (La Memoria a Corto Plazo):**
   - Almacena los `last_seen_id` y los hashes de deduplicación.
   - Respuesta en microsegundos. Imprescindible para que el scraper no espere a la base de datos.
2. **Bloom Filters (Filtros Probabilísticos):**
   - Usaremos filtros de Bloom en Redis para comprobar existencia de URLs/IDs masivamente con un coste de memoria ridículo (bits por entrada).
   - *Pregunta:* "¿Hemos visto esta URL antes?" -> *Respuesta:* "No" (100% seguro) o "Probablemente sí".

MÓDULO 2.2.1: INFRAESTRUCTURA GEOINT DE "TIERRA QUEMADA" (EL MAPS DRILLER)

TESIS DE INVERSIÓN: El mercado de VO no está en internet; está en el suelo. El 40% del inventario europeo vive en negocios que no saben digitalizarse. Nuestro objetivo no es encontrarlos; es **inventariarlos antes de que ellos mismos sepan lo que tienen**. **KPI DE ÉXITO:** Coste de Adquisición de Dato (CAC) = 0,0001€. Cobertura Física = 99.9% (incluyendo zonas rurales).

1. ARQUITECTURA DE DATOS ESPACIALES (EL TABLERO DE AJEDREZ)

Olvídate de "cuadrículas de km". Eso es geometría euclidiana básica. Para dominar Europa, necesitamos topología jerárquica.

DECISIÓN TÉCNICA: H3 (UBER'S HEXAGONAL HIERARCHICAL SPATIAL INDEX)

- **Por qué:** Los hexágonos (H3) tienen vecinos equidistantes. Las cuadrículas no. Para calcular rutas logísticas y densidades de "scraping", H3 es matemáticamente superior.
- **Implementación:**
  - **Nivel H3-6 (Area ~36 km²):** Unidad de "Exploración".

- **Nivel H3-9 (Area ~0.1 km²):** Unidad de "Ataque Urbano".
- **Base de Datos: PostGIS** con indexación nativa H3. No guardamos "puntos"; guardamos **Células de Hexágono** con estado (Explorado/Virgen/Saturado).

2. FASE 1: LA CAPA BASE "ZERO-COST" (OVERTURE FOUNDATION)

No pagaremos el "impuesto Google" por datos que ya son libres. Google Maps API es para validación quirúrgica, no para pesca de arrastre.

- **La Fuente: Overture Maps Foundation** (Dataset `places` - 2.3 Billones de POIs globales). Respaldado por Amazon, Meta, Microsoft.
- **Protocolo de Extracción Masiva (DuckDB on Bare Metal):**
  - Montamos un pipeline en el **AX102** usando **DuckDB** para ingerir los archivos Parquet (S3) sin descargarlos completos.

Query de Extracción Forense:
SQL

```
SELECT id, names, categories, websites, socials, geometry

FROM overture_places

WHERE bbox.xmin > -10 AND bbox.xmax < 30 -- Europa Oeste

AND (

  categories.main = 'automotive_dealer' OR

  categories.alternate LIKE '%used_car%' OR

  categories.alternate LIKE '%auto_repair%' OR

  categories.alternate LIKE '%junkyard%' -- Desguaces: Oro puro

)
```

-
- **Resultado:** Obtenemos ~1.2 Millones de puntos de venta base en Europa en **15 minutos** a **Coste 0€**.

3. FASE 2: EL "GHOST WALKER" (SCRAPING GOOGLE SIN API)

Aquí es donde aplicamos la ingeniería inversa. Overture nos da la base, pero Google tiene la verdad fresca (fotos de ayer, reseñas de hoy). La API oficial es una trampa financiera. La evitaremos.

PROTOCOLO "SIDE-CHANNEL SCRAPING": En lugar de la API, atacamos la interfaz pública de Google Maps (`google.com/maps/search`) simulando tráfico humano indetectable.

1. **Generación de Consultas H3:**

    - El sistema toma un hexágono H3. Genera el centroide Lat/Lon.
    - Construye la URL de búsqueda: `https://www.google.com/maps/search/concesionario/@{lat},{lon},15z`.
2. **Extracción Visual del DOM (Playwright + Stealth):**
    - Navegamos a la URL. Google renderiza el "Side Panel" con los negocios.
    - **Extracción de Datos Ocultos (CID):** Extraemos el `data-cid` (Customer ID interno de Google). Es la huella digital inmutable del negocio.
    - **Validación de Existencia:** Si el negocio de Overture no sale en Maps, lo marcamos como "Zombi" (Cerrado).
3. **Minería de "The Knowledge Graph":**
    - Extraemos el JSON-LD (Linked Data) incrustado en el HTML. Ahí está el teléfono, el horario y las "Popular Times" (horarios de afluencia) sin hacer clic.

4. FASE 3: INTELIGENCIA VISUAL (EL "STREET RECON")

La web miente. Las fotos no. Necesitamos saber si "Talleres Paco" tiene 2 coches o 200.

PIPELINE DE VISIÓN FORENSE (LOCAL - GPU/CPU):

1. **Ingesta de Imágenes GMB (Google My Business):**
    - Descargamos las últimas 20 fotos etiquetadas como "Owner" (Dueño) y "Exterior".
2. **Análisis de Escena (YOLOv8 Fine-Tuned):**
    - Entrenamos un modelo YOLOv8-Nano (ultrarrápido) específicamente para detectar:
        - *Car Density:* ¿Coches aparcados en batería? -> Concesionario. ¿Coches en elevador? -> Taller.
        - *Signage:* ¿Carteles de precios en parabrisas? (A4 papers).
        - *License Plates:* ¿Matrículas rojas/verdes (profesionales) o tapadas?
3. **Extracción de Texto (PaddleOCR):**
    - Si detecta cartel -> OCR.
    - **Objetivo:** Extraer móviles personales escritos a mano en el cartel que NO están en la ficha de Google. **Esto es el contacto directo con el dueño.**

5. FASE 4: VIGILANCIA PERIMETRAL (EL "GAP FILLING")

Overture y Google tienen lagunas. Los negocios nuevos o clandestinos no salen.

ALGORITMO DE TRIANGULACIÓN INVERSA:

1. **Detección de Vacíos:** Si un Hexágono H3 en zona urbana densa tiene 0 resultados, es una anomalía estadística.
2. **Sonda de "Street View" (GSV Hack):**
    - Lanzamos una petición a la API no documentada de metadatos de Street View en esas coordenadas.

- Si la imagen es reciente (<6 meses) y la segmentación semántica de la imagen (usando SegFormer) detecta "Alta densidad de vehículos", **marcamos la zona para revisión humana o scraping profundo**.
- *Nota:* Esto nos permite encontrar campas ilegales o concesionarios temporales.

MÓDULO 2.2.2: INFRAESTRUCTURA "SOCIAL HUNTER" (INFILTRACIÓN EN REDES SOCIALES)

OBJETIVO: Extraer el "Inventario Sombra" (Shadow Inventory). Coches que se venden en Facebook Marketplace, Instagram Stories y Grupos de Telegram antes de llegar a los portales. **ENEMIGO:** Meta (Facebook/Instagram) Integrity Systems, Datadome, Shape Security. **ESTRATEGIA:** Mimetismo Humano Distribuido + Intercepción de GraphQL.

A. INFRAESTRUCTURA DE IDENTIDAD (THE FARM)

Para scrapear Meta, necesitas ser "alguien". Un script anónimo no ve nada.

1. **Gestión de Activos Digitales (Cuentas "Añejas"):**
    - **Adquisición:** No creamos cuentas (nos cazan por huella de creación). Adquirimos **"Aged Accounts" (PZ > 2 años)** con actividad real previa (cookies de navegación, historial de likes).
    - **Protocolo de Calentamiento (The Warming Protocol):**
        - Día 1-3: Login pasivo. Scroll aleatorio en el feed. 0 búsquedas.
        - Día 4-6: Interacción ligera (Like a fotos de gatos, unirse a grupos de barrio).
        - Día 7: Primera búsqueda en Marketplace.
        - *Herramienta:* **Scripts de Puppeteer-Extra-Stealth** que ejecutan estas acciones de forma aleatoria ("ruido humano") las 24h para mantener el "Trust Score" alto.
2. **Aislamiento de Huella (Fingerprint Isolation):**
    - Cada cuenta vive en un **Contenedor de Navegador Estanco** (usando perfiles de **Adspower** o **Dolphin{anty}** controlados vía API).
    - **Parámetros Únicos por Cuenta:**
        - Canvas Hash & WebGL Noise (Único por perfil).
        - Fuentes instaladas & Rects del navegador.
        - Hardware Concurrency (Cores simulados) y RAM simulada.
    - **Red:** Proxy 4G Móvil Dedicado (IP Rotativa pero con "Sticky Session" durante la operación). *Jamás* dos cuentas comparten la misma IP simultáneamente.

B. FACEBOOK MARKETPLACE: INTERCEPCIÓN GRAPHQL (GOD MODE)

Scrapear el HTML de Facebook (`<div class="x1y...">`) es suicida. Las clases cambian cada compilación. Atacaremos el cerebro: **GraphQL**.

1. **Vector de Ataque (La API Oculta):**
    - Facebook carga los datos mediante peticiones `POST /api/graphql`.
    - **El Payload:** Identificamos la query `MarketplaceSearchQuery`.
    - **La Táctica:**
        - El navegador "Zombie" navega a Marketplace.
        - Inyectamos un script (vía Chrome DevTools Protocol) que intercepta la respuesta JSON de GraphQL *antes* de que se renderice en pantalla.
        - **Resultado:** Obtenemos datos puros: `listing_id`, `price`, `seller_id`, `creation_time` (segundo exacto), `location` (lat/lon real, no aproximada).
2. **Geolocalización Fantasma (Teletransporte):**
    - Marketplace muestra resultados basados en la ubicación del usuario.
    - **El Hack:** Inyectamos coordenadas GPS falsas en el navegador (`navigator.geolocation`) y modificamos las cookies de sesión `c_user` y `datr` para simular saltos geográficos.
    - **Barrido:** Saltamos de centroide urbano en centroide urbano (Grid de 50km) para cubrir el mapa de Europa sin dejar zonas muertas.
3. **Extracción de Contacto (El Muro):**
    - FB oculta el teléfono. El botón "Enviar Mensaje" es una trampa (si mandas 50 mensajes, te banean).
    - **Solución Forense:**
        - El 40% de los vendedores pone el teléfono en la descripción o "camuflado" (`6-seis-6...`). -> **Regex NLP**.
        - El 30% lo pone en una foto. -> **Pipeline de Visión (Módulo 1.2)** para leer píxeles.
        - **Linkage:** Si no hay teléfono, extraemos el `seller_id` y buscamos ese ID en nuestra base de datos de fugas anteriores o perfiles públicos cruzados (OSINT) para desanonimizar al vendedor.

C. INSTAGRAM & TIKTOK: MINERÍA DE LO EFÍMERO (STORIES)

El "Alpha" está en las Stories. El dealer sube la "Gema" a Stories: *"Acaba de entrar, 15.000€"*. A las 2 horas está vendido. Si esperas al feed, llegas tarde.

1. **Monitorización de Perfiles Objetivo (Watchlist):**
    - Mantenemos una lista de 50.000 perfiles de dealers identificados (via Módulo 2.2.1).
    - **Frecuencia:** Checkeo cada 4 horas.
2. **Pipeline de Extracción de Stories (Sin API):**
    - Usamos **Instagrapi** (Python) con cuentas "Burner" (desechables) para descargar el media.
    - **Análisis Multimodal:**
        - **Frame Extraction:** Sacamos 1 frame por segundo del video.

        - **Modelo VLM (Vision Language Model):** Preguntamos a **Moondream2**: *"¿Hay un coche? ¿Hay un precio sobreimpreso? ¿Hay texto de oferta?"*.
        - **Transcripción de Audio (Whisper):** Si el dealer habla (*"Este Golf está impecable"*), transcribimos a texto y lo indexamos.
3. **Hashtag Listener (La Red de Arrastre):**
    - Monitoreo de tags de venta rápida: `#urgente`, `#liquidación`, `#export`.
    - Detección de patrones visuales: Foto de un coche con el texto "VENDIDO" tachado -> Lo usamos para entrenar nuestro modelo de precios (sabemos que *ese* coche a *ese* precio voló, ergo el precio era bueno).

D. TELEGRAM & WHATSAPP GROUPS (DARK POOLS)

Muchos tratos B2B ocurren en grupos cerrados de compraventa.

1. **Infiltración de Agentes (Userbots):**
    - Usamos la librería **Telethon** (Python) para conectar cuentas de Telegram reales.
    - Nos unimos a cientos de grupos de "Compraventa Coches Europa", "Importación Alemania", etc.
2. **Escucha Pasiva y Parsing:**
    - Escaneamos el chat en tiempo real.
    - **Parser de Texto Desestructurado (DeepSeek):** Convertimos un mensaje de chat *"Hola vendo audi a4 2018 100k km 18mil"* en un JSON estructurado de vehículo.
    - **Alerta Sniper:** Si entra una oferta válida, el sistema la inyecta en el dashboard antes de que nadie conteste en el chat.

MÓDULO 2.3: NORMALIZACIÓN FORENSE Y DICCIONARIO MAESTRO ("EL CRISOL")

OBJETIVO: Convertir el caos lingüístico y estructural de 6 idiomas y 50 portales en un **Estándar Financiero Unificado (CARDEX_STD)**.

FILOSOFÍA: "El vendedor miente. El VIN no".

2.3.1. EL DICCIONARIO MAESTRO VECTORIAL (SEMÁNTICA, NO TRADUCCIÓN)

Olvídate de tablas de traducción SQL (if 'techo' then 'sunroof'). Eso es frágil. Usaremos **Inteligencia Vectorial Local**.

LA ARQUITECTURA "SEMANTIC CORE":

1. **Modelo de Embeddings (Local):**
    - Desplegamos **bge-m3** o **paraphrase-multilingual** (Modelos SOTA para similitud semántica) en el Nodo 03 (CPU).
    - Transformamos nuestra lista maestra de 500 equipamientos (XENON, NAVI, LEATHER) en vectores matemáticos.
2. **Ingesta de Caos:**
    - Entra texto sucio: *"asientos d piel calefactados y con memoria"*.
    - Entra texto alemán: *"ledersitze sitzheizung memory"*.
    - El modelo vectoriza la entrada y busca los vectores más cercanos en nuestro Diccionario Maestro.
    - **Resultado:** [SEAT_LEATHER, SEAT_HEATED, SEAT_MEMORY].
    - **Ventaja:** Funciona con faltas de ortografía, abreviaturas ("S/R", "PDC") y jerga de taller sin programar ni una sola regla if/else.

2.3.2. INYECCIÓN DE VERDAD (VIN DECODING & FACTORY SPECS)

El vendedor dice "Full Equip". Nosotros verificamos la hoja de fabricación.

LA ESTRATEGIA DE "ADN DEL COCHE":

1. **Minería de Datos de Fabricante (OEM Scraping):**
    - Muchos portales de recambios (Partslink24, portales rusos de VIN) permiten obtener la lista de **PR Codes** (Códigos de producción) gratis o por céntimos.
    - **Táctica:** Cuando detectamos un VIN, lanzamos un agente a estas fuentes "grises".
2. **Validación Cruzada (The Lie Detector):**
    - El anuncio dice: "BMW M3".
    - El VIN Decoder dice: "BMW 320d con Paquete M".
    - **Acción:** El sistema reescribe el título del anuncio en nuestra base de datos a "BMW 320d (Fake M3)" y alerta al usuario. **Esto es Alpha puro: evitar que el cliente compre un coche maquillado.**

2.3.3. NORMALIZACIÓN FINANCIERA (IVA Y RÉGIMEN FISCAL)

Detectar si un coche es Neto o Bruto es la diferencia entre ganar un 20% o perder un 5%. No podemos fallar.

EL ALGORITMO "TAX HUNTER":

1. **Detección NLP de Régimen de IVA:**
    - No confiamos en la casilla "IVA Deducible" del portal (los dealers se equivocan el 30% de las veces).
    - Analizamos la descripción con un modelo de clasificación de texto (BERT-Tiny local):
        - *Keywords Positivas:* "MwSt ausweisbar", "Netto export", "TVA recuperable".
        - *Keywords Negativas:* "Im Kundenauftrag" (Venta de cliente a cliente - NO hay garantía ni IVA), "§25a" (Régimen de margen alemán).
2. **Cálculo de Precio Real:**
    - Normalizamos todo a **Precio Neto Exportación**.

- Si es REBU (Régimen Especial Bienes Usados), el sistema marca vat_reclaimable: false.

2.3.4. DEDUPLICACIÓN PERCEPTUAL (RASTREO DE "COCHES FANTASMA")

Un mismo coche está en Mobile.de (por el dueño), en Autoscout24 (por un intermediario) y en Facebook (por el sobrino). Si mostramos 3 coches, fallamos.

EL PROTOCOLO "PHASH" (PERCEPTUAL HASHING):

1. **Huella Visual:** Generamos un hash de la imagen principal del coche que resiste cambios de tamaño, marcas de agua o recortes.
2. **Huella de Datos:** Hash de (Modelo + Año + Km (redondeado al 1000) + Código Postal).
3. **Fusión:** Si Distancia(HashA, HashB) < Umbral, el sistema fusiona los registros.
   - **Efecto:** El usuario ve: "1 Coche encontrado en 3 fuentes".
   - **Alpha:** Vemos arbitraje de precios del mismo coche. "En Mobile.de está a 20k, en Facebook el dueño lo tiene a 18.5k". **COMPRAR EN FACEBOOK.**

RESUMEN DE EJECUCIÓN Y COSTES (CHECKLIST 2.3)

ELEMENTO	ACCIÓN TÉCNICA	COSTE ESTIMADO
Vector Engine	Despliegue de **Qdrant** (Vector DB) + Modelo bge-m3 en Nodo 03.	**0€** (Uso de CPU local)
VIN Decoder	Script de scraping contra portales de recambios públicos/rusos.	**0€** (Si usamos proxies rotativos ya pagados)
Tax AI	Entrenamiento de DistilBERT con dataset propio de 10k descripciones financieras.	**0€** (Entrenamiento único en local)
Deduplicación	Librería ImageHash (Python) ejecutándose en el pipeline de ingesta.	**0€** (Cómputo puro)
Mantenimiento	Actualización mensual de la base de datos de modelos/versiones nuevos.	**Horas de Ingeniero**

COSTE TOTAL DEL MÓDULO: 0€ de OPEX adicional. Solo consumo de CPU y electricidad del cluster Hetzner ya pagado.

MÓDULO 3.0: INTELIGENCIA DE MERCADO Y MOTOR DE ARBITRAJE (EL CEREBRO)

CLASIFICACIÓN: CRÍTICO / CORE BUSINESS

OBJETIVO: DETERMINAR EL "NET LANDED COST" (COSTE REAL FINAL) CON PRECISIÓN DE CÉNTIMO EN <200ms.

FILOSOFÍA: "El precio del anuncio es una mentira. El margen neto es la verdad."

3.1. ARQUITECTURA DEL "NET LANDED COST" (NLC)

La competencia (AutoScout, Indicata) muestra precios brutos. Eso es inútil para un profesional. CARDEX calcula la **Ecuación Maestra del Arbitraje**.

$$NLC = (P_{origen} - IVA_{reclamable}) + T_{logística} + T_{impuestos} + T_{gestión} + T_{reacondicionamiento}$$

3.1.1. EL MOTOR FISCAL (LA MATRIZ DE 6 PAÍSES)

El algoritmo más complejo y valioso de la plataforma. No usa IA (la IA alucina números). Usa Motores de Reglas Deterministas en Python/Rust.

A. DETECCIÓN DE RÉGIMEN DE IVA (EL FILTRO BINARIO)

- **Problema:** Un coche de 30.000€ en Alemania puede costar 30.000€ (REBU/Margen) o 25.210€ (Neto/IVA Deducible) para un profesional español.
- **Solución Técnica:**
  - **Regex Forense:** Analizamos la descripción con patrones estrictos en el idioma origen.
  - *Alemania:* MwSt. ausweisbar | Netto -> **DEDUCIBLE**.
  - *Francia:* TVA récupérable -> **DEDUCIBLE**.
  - *Negativos:* §25a (DE), Marge (FR), Particular -> **NO DEDUCIBLE**.
  - **Validación Cruzada:** Si el vendedor es "Private Seller", forzamos vat_deductible = False aunque el texto diga lo contrario.

B. CALCULADORA DE IMPUESTOS DE MATRICULACIÓN (CO2)

- **España (IEDMT):**

    - Cruzamos el VIN con nuestra base de datos técnica (Módulo 2.3) para obtener el CO2 WLTP exacto.
    - Aplicamos las tablas de Hacienda vigentes (0%, 4.75%, 9.75%, 14.75%) sobre el **Valor Ganvam** (Tablas de depreciación oficial), no sobre el precio de compra.
- **Francia (Malus Écologique):**
    - El algoritmo calcula el recargo exponencial basado en gramos de CO2.
    - *Alerta Roja:* Si el Malus supera los 10.000€ (común en deportivos), el sistema marca el coche como **"TÓXICO PARA FRANCIA"** y bloquea la recomendación para clientes franceses.
- **Holanda (BPM):**
    - Calculamos el "Rest-BPM" (Impuesto residual) deducible al exportar desde Holanda (Dinero que el gobierno devuelve al dealer). Esto convierte a Holanda en una mina de oro para la exportación que nadie más calcula.

3.2. EL "INDICATA-KILLER": PRICING DINÁMICO DE ALTA FRECUENCIA

Los valoradores actuales (Eurotax, DAT) usan datos de hace 30 días. CARDEX usa datos de hace 30 segundos.

3.2.1. VALORACIÓN DE LIQUIDEZ (VELOCITY)

No importa cuánto vale el coche, importa **cuándo** se vende.

- **Algoritmo "Survival Analysis":**
    - Para cada modelo (ej: BMW Serie 3 G20), analizamos la curva de desaparición de anuncios en la región destino.
    - **KPI:** Days-To-Sale (DTS).
    - **Clasificación:**
        - *DTS < 15 días:* **LIQUIDEZ EXTREMA (A+)**. Se puede pagar un sobreprecio.
        - *DTS > 90 días:* **ACTIVO TÓXICO (F)**. Penalizamos la valoración un 15% por coste financiero de stock.

3.2.2. SELLER DESPERATION INDEX (SDI) - EL PSICÓLOGO ALGORÍTMICO

Detectamos cuando un vendedor necesita efectivo *ya*.

- **Señales de Pánico (Weighted Score):**
    1. **Caída de Precio Acelerada:** Bajó un 5% ayer y un 3% hoy.
    2. **Cambio de Título:** Añadió "URGE" o "OFERTA" al título.
    3. **Hora de Publicación:** Publica un domingo a las 23:00 (Particular) o fin de mes (Dealer cerrando objetivos).
    4. **Enriquecimiento Visual:** Subió mejores fotos después de 2 semanas sin vender.

- **Acción:** Si SDI > 80/100, enviamos alerta **"VENDEDOR SANGRANDO"**. Oportunidad de oferta agresiva (-20% del precio marcado).

3.3. EL SCORING DE OPORTUNIDAD ("EL SEMÁFORO")

Condensamos millones de datos en una decisión simple para el humano.

EL ALGORITMO "CARDEX ALPHA SCORE" (0-100):

Un modelo de *Ensemble* (XGBoost) entrenado con datos históricos de "Coches que generaron margen real".

FACTORES DE PONDERACIÓN:

1. **Margen Neto Proyectado (40%):** (Precio Venta Destino Estimado - NLC). Si es < 5%, Score = 0.
2. **Velocidad de Venta (30%):** ¿Lo venderé rápido?
3. **Fiabilidad del Dato (15%):** ¿Tenemos VIN? ¿El vendedor es verificado? (Si es una estafa probable, Score = 0).
4. **Escasez Regional (15%):** ¿Hay 50 iguales en mi zona o es el único?

RESULTADO VISUAL EN UI:

- **90-100 (DIAMANTE):** Margen >20%, Venta <15 días. **COMPRA AUTOMÁTICA.**
- **75-89 (ORO):** Margen >12%. Buena compra.
- **<50 (PLOMO):** Ignorar. Ruido.

3.4. ARBITRAJE LOGÍSTICO Y COSTES OCULTOS

El transporte no es una tarifa plana. Es un mercado dinámico.

LA CALCULADORA GEO-ESPACIAL:

1. **Ruteo Real:** Usamos **OSRM (Open Source Routing Machine)** localmente para calcular km reales de carretera, no línea recta.
2. **Matriz de Costes Dinámica:**
   - *Ruta Caliente (ej: Frankfurt -> Madrid):* Coste bajo (retornos de camiones vacíos).
   - *Ruta Fría (ej: Sicilia -> Brest):* Coste alto (requiere trasbordos).
3. **Estimación de Daños (Reacondicionamiento):**
   - La IA de visión (Módulo 2.5) detectó "roce en parachoques".
   - El sistema suma automáticamente **150€ (Smart Repair)** al NLC.
   - Resultado: No hay sorpresas financieras al recibir el coche.

3.5. INFRAESTRUCTURA DE CÓMPUTO DEL MÓDULO 3

Aquí definimos dónde corre esta matemática pesada.

- **Base de Datos Analítica (ClickHouse):**
  - Almacena trillones de puntos de datos de precios históricos.
  - Permite calcular la "Media de Mercado" y la "Desviación Estándar" de 50.000 Golfs en **15 milisegundos**.
  - *Consulta Típica:* SELECT avg(price) FROM market_history WHERE model='Golf' AND year=2020 AND country='ES'.
- **Motor de Inferencia (Python/Pandas Polars):**
  - Ejecutamos los cálculos de NLC en memoria usando **Polars** (librería ultra-rápida en Rust para Python) en el Nodo 01.
  - Capaz de re-calcular el margen de 100.000 oportunidades cada vez que cambia el tipo de cambio divisa (CHF/EUR) o una regulación fiscal.

RESUMEN DE EJECUCIÓN Y COSTES (CHECKLIST 3.0)

ELEMENTO	ACCIÓN TÉCNICA	COSTE ESTIMADO
Motor Fiscal	Codificación de reglas tributarias de los 6 países (Python puro).	**0€** (Desarrollo interno)
Base de Datos OLAP	Despliegue de **ClickHouse** en Nodo 01 (NVMe).	**0€** (Infra ya pagada)
Modelo Pricing	Entrenamiento de **XGBoost** con dataset histórico (2 años) de precios.	**0€** (Cómputo local)
Geo-Routing	Despliegue de **OSRM** Dockerizado con mapas de Europa.	**0€** (Open Source)
API Externa	Integración API tasas de cambio (OpenExchangeRates)	**~10€/mes**

COSTE TOTAL DEL MÓDULO: Marginal. El valor es intelectual (los algoritmos), no de infraestructura.

MÓDULO 4.0: EL HISTORIAL UNIFICADO (LA VERDAD GRATIS)

CLASIFICACIÓN: INTELIGENCIA DE ACTIVOS / OSINT **OBJETIVO:** GENERAR UN "SHADOW REPORT" (INFORME SOMBRA) SUPERIOR A CARFAX SIN PAGAR A CARFAX. **FILOSOFÍA:** "El coche tiene memoria digital. Solo hay que saber interrogar a internet."

4.1. CAPA 1: ARQUEOLOGÍA DIGITAL (EL RASTRO INBORRABLE)

Cuando un coche se vende, deja huellas. Cuando se choca, deja huellas. Cuando se subasta, deja huellas. Los vendedores intentan borrarlas. Nosotros las tenemos cacheadas.

4.1.1. EL MOTOR DE "VIN DORKING" (BÚSQUEDA PROFUNDA)

No buscamos en bases de datos estructuradas. Buscamos en la basura de internet.

1. **Rastreo de Archivos de Subastas (Salvage & Wrecks):**
    - Los coches accidentados de EEUU/Europa del Este/Alemania acaban en portales como `bidfax.info`, `poctra.com`, `autoastat.com`.
    - **El Bot:** Scrapea diariamente estos sitios y guarda: `VIN + FOTOS DEL GOLPE + KM`.
    - **El Match:** Cuando ese coche aparece "reparado" en Mobile.de 6 meses después como "Libre de accidentes", CARDEX cruza el VIN.
    - **Resultado:** Mostramos al cliente la foto del coche destrozado gratis. **Valor infinito.**
2. **Búsqueda Inversa de Imágenes (Visual Trace):**
    - Si el vendedor tapa la matrícula o no pone el VIN, usamos la foto.
    - **Algoritmo:** Hash Perceptual (pHash) contra nuestra base de datos histórica de 12M de coches.
    - **Detección:** "¿Este coche se vendió hace 3 meses en Polonia con 200.000km y ahora está en España con 120.000km?".
    - **Alerta:** "DETECTADO AFEITADO DE KM (Fuente: Anuncio previo ID #88442)".

4.1.2. MINERÍA DE "LIBROS DE MANTENIMIENTO" (OCR TÁCTICO)

Los vendedores suben fotos del libro de revisiones para dar confianza. **Grave error.**

1. **Pipeline de Visión (Local - PaddleOCR):**
    - Detectamos fotos que parecen documentos (sellos, fechas, firmas).
    - Extraemos: `Fecha`, `Kilometraje`, `Nombre del Taller`.
2. **Verificación de Coherencia:**
    - Si la revisión de 2022 marca 140.000km y el cuadro marca 130.000km -> **FRAUDE DETECTADO.**
    - Si el sello del taller es siempre el mismo con la misma tinta y firma en 5 años -> **POSIBLE FALSIFICACIÓN.**

4.2. CAPA 2: INGENIERÍA INVERSA DE FUENTES GUBERNAMENTALES (HACKING INSTITUCIONAL)

Los gobiernos tienen los datos. Algunos los venden, otros los regalan si sabes pedirlo. No pagaremos tasas si podemos evitarlo.

4.2.1. ESPAÑA (DGT) - EL HACK DEL "INFORME REDUCIDO"

- **La Vía Oficial:** Pagar tasa (caro y lento).
- **La Vía CARDEX (Automatización de Ciudadano):**
    - El "Informe Reducido" es gratuito. Dice: Fecha matriculación, Combustible y **ESTADO (Incidencias)**.
    - **Implementación:** Bot con Certificado Digital Cualificado (FNMT) que consulta masivamente vía headless browser la sede electrónica.
    - **Dato Crítico:** Si sale "Incidencias: SÍ" (Embargo/Precinto), descartamos el coche automáticamente. Coste: 0€.

4.2.2. REINO UNIDO (DVLA) - LA MINA DE ORO ABIERTA

- La API del DVLA (MOT History) es **GRATUITA y PÚBLICA**.
- **Dato:** Historial completo de ITVs, fallos técnicos y kilometraje anual exacto.
- **Uso:** Para importaciones UK->Europa, tenemos el historial clínico perfecto a coste cero.

4.2.3. PAÍSES BAJOS (RDW) - OPEN DATA PURO

- La RDW ofrece una API abierta (`Opendata.rdw.nl`).
- **Datos:** Estado de robo, fecha de exportación, características técnicas exactas, catalogación de precios.
- **Estrategia:** Ingesta diaria completa del parque holandés. Si un coche viene de Holanda, sabemos más que el vendedor.

4.2.4. ITALIA (IL PORTALE DELL'AUTOMOBILISTA)

- Web pública para verificar la "Ultima Revisione".
- **Scraper:** Introduce matrícula -> Obtiene fecha y KM de la última ITV.
- **Coste:** 0€. Solo coste de resolver el Captcha.

4.3. CAPA 3: INTELIGENCIA DE FABRICANTE (OEM LEAKS)

Los fabricantes saben cómo salió el coche de fábrica.

4.3.1. DECODIFICADOR VIN "GRAY HAT"

- **Fuente:** Portales de recambios (Partslink24, Catálogos Rusos de ETKA/EPC).
- **Método:** Estos portales permiten meter el VIN para buscar piezas. Al hacerlo, devuelven la "Data Card" (Equipamiento de fábrica).
- **Extracción:**
    - Fecha de producción exacta (clave para impuestos).
    - Motor y transmisión real (evita errores de "es un 150cv" cuando es un "116cv").
    - Equipamiento original (detecta si le han quitado piezas o si el "Pack M" es falso).

4.4. CAPA 4: EL ALGORITMO DE INFERENCIA (CUANDO NO HAY DATOS)

Si no hay registros, usamos estadística predictiva.

4.4.1. MODELO DE PROBABILIDAD DE FRAUDE (ML)

Entrenamos un modelo **XGBoost** con millones de coches históricos.

- **Inputs:** Modelo, Motor, Antigüedad, País Origen, Tipo de Vendedor (Particular vs Compraventa).
- **Lógica:** "Un Mercedes Clase E Diesel de 4 años en Alemania (Ex-Taxi probable) con 60.000km es estadísticamente imposible (Anomalía > 3 sigma)".
- **Output:** Score de Riesgo de Kilometraje (0-100). Aunque no tengamos el dato real, avisamos: **"Riesgo Estadístico de Afeitado: ALTO (95%)"**.

4.5. CAPA 5: INTEGRACIÓN DE PAGO (SOLO BAJO DEMANDA)

Solo pagamos cuando el usuario paga.

4.5.1. EL "BRIDGE" API (CARFAX / CARVERTICAL)

- Mantenemos una integración técnica lista.
- **Botón en UI:** "Obtener Informe Certificado Oficial (15€)".
- **Margen:** Nosotros lo compramos a precio mayorista (volumen) por 8€ y lo vendemos a 15€.
- **Estrategia:** Usamos el informe gratuito (Capa 1-4) para convencer al usuario de que el coche merece la pena, y luego le vendemos la certificación final.

MÓDULO 5.0: EL SISTEMA OPERATIVO B2B (ARQUITECTURA DE EJECUCIÓN)

ESTADO: ESPECIFICACIÓN DE INGENIERÍA **OBJETIVO:** CREAR UN ECOSISTEMA "STICKY" CON COSTE OPERATIVO MARGINAL CERCANO A CERO. **HARDWARE:** CLUSTER HETZNER AX102 (CPU RYZEN 9 7950X3D).

5.1. LA INTERFAZ: "MESA DE TRADING" (ARQUITECTURA FRONTEND)

No usamos plantillas. Construimos una aplicación de baja latencia diseñada para la velocidad de decisión.

5.1.1. STACK TECNOLÓGICO (EL MOTOR GRÁFICO)

- **Framework: Next.js 14 (App Router)**.
    - *Por qué:* Server-Side Rendering (SSR) para carga inicial instantánea (<300ms) y SEO en perfiles públicos de dealers.
- **Gestión de Estado (La Memoria): TanStack Query (React Query)** + **Zustand**.
    - *Lógica:* No hacemos "fetching" cada vez. Mantenemos una caché local en el navegador del cliente. Si el dealer cambia de pestaña y vuelve, los datos están ahí instantáneamente. Cero carga al servidor.
- **Grid de Datos (El Excel Killer): AG Grid Enterprise**.
    - *Capacidad:* Puede renderizar 100.000 filas en el navegador con virtualización (solo pinta lo que se ve en pantalla). Permite filtros, pivotes y agrupación en el cliente.
    - *Coste:* Licencia ~700€/año (o versión Community gratis inicial). *Decisión: Community mode hackeada con plugins propios para empezar.*

5.1.2. CONECTIVIDAD EN TIEMPO REAL (WEBSOCKETS)

- **Tecnología: Socket.io** sobre servidor **Node.js** (separado de Next.js) en el Nodo 01.
- **Protocolo:**
    1. El Backend detecta un cambio de precio en ClickHouse.
    2. Publica evento en **Redis Pub/Sub**.
    3. El servidor Socket.io empuja el cambio solo a los clientes conectados interesados.
- **Impacto:** El dealer ve el precio cambiar en su pantalla en **tiempo real** sin recargar.
- **Coste de Infra:** Uso de RAM en el servidor (despreciable para <10k usuarios concurrentes).

5.2. EL CRM INVISIBLE (INGENIERÍA DE AUTOMATIZACIÓN)

El dealer no tecleará datos. El sistema los "aspira".

5.2.1. MOTOR DE INGESTA "MAGIC LINK" (PIPELINE)

El Flujo Técnico:

1. **Input:** Dealer pega URL de Wallapop/Mobile.de.
2. **Scraper Táctico (Ejecución):**
    - El backend lanza un contenedor efímero de **Crawlee** (Node.js) en el Nodo 02.
    - *Coste:* 0,0001€ de CPU por ejecución.
3. **Parser Estructural:** Extrae el JSON-LD incrustado en el HTML (datos estructurados de Google) para obtener Marca, Modelo, Año y Precio con 100% precisión, sin depender de selectores CSS frágiles.
4. **Descarga de Media:** Descarga las imágenes a la RAM, las comprime a **AVIF** (formato open source de alta compresión, 50% menos peso que JPG) usando **libavif** y las guarda en disco.

5.2.2. GESTIÓN DOCUMENTAL (OCR & PDF GENERATION)

El Stack de Procesamiento:

- **OCR de DNI/Ficha Técnica: PaddleOCR** (Modelo `ch_ppocr_v3`).
    - *Viabilidad:* Corre en la CPU del Ryzen 9 a ~200ms por página. No requiere GPU.
    - *Coste:* 0€.
- **Generador de PDF: React-PDF** (Renderizado en servidor).
    - *Lógica:* Usamos componentes React para diseñar el contrato y lo renderizamos a PDF en el backend.
    - *Firma Digital:* Integración con API de **DocuSign** (Coste alto) o implementación propia con **eIDAS** usando certificados digitales (Coste

desarrollo alto, coste por uso 0€). *Decisión Fase 1: Implementación propia de firma simple con traza de auditoría (IP, Timestamp, Hash).*

5.3. AGENCIA DE MARKETING AUTOMATIZADA (IA GENERATIVA)

Aquí es donde la mayoría quiebra por costes de API. Nosotros no.

5.3.1. ESTUDIO FOTOGRÁFICO IA (REMASTERIZACIÓN LOW-COST)

El dealer sube fotos horribles. Las arreglamos.

1. **Eliminación de Fondo:** Librería **Rembg** (basada en modelo U2Net).
    - *Ejecución:* Corre en CPU (ONNX Runtime). Tarda ~1.5s por foto en el Ryzen 9.
    - *Coste:* 0€.
2. **Super-Resolución (Upscaling):** Modelo **RealESRGAN** (ncnn/Vulkan).
    - *Ejecución:* Corre en CPU optimizada.
    - *Coste:* 0€.
3. **Generación de Fondo (El Truco):**
    - *Problema:* Generar un fondo con Stable Diffusion en CPU tarda 30 segundos. Inviable para 20 fotos.
    - *Solución Híbrida:* Usamos fondos pre-renderizados de alta calidad (Studio, Paisaje) y componemos el coche encima usando **OpenCV** (mezcla de capas y corrección de color automática).
    - *Resultado:* "Look" profesional en 2 segundos por foto a coste cero. Solo usamos IA generativa pura si el usuario paga créditos extra.

5.3.2. COPYWRITING AUTOMÁTICO (LLM LOCAL)

- **Tecnología: Qwen-2.5-7B-Instruct** (Quant Q4_K_M) corriendo en **Ollama** (Nodo 03).
- **Prompt Engineering:**
    - Input: JSON del coche (`{"model": "BMW X5", "extras": ["techo", "cuero"]}`).
    - Prompt del Sistema: *"Eres un vendedor de coches de lujo experto. Escribe una descripción persuasiva de 200 palabras enfatizando la exclusividad."*
- **Rendimiento:** El Ryzen 9 genera esto en ~4 segundos.
- **Coste:** Electricidad.

5.3.3. MULTIPOSTING (RPA - ROBOTIC PROCESS AUTOMATION)

Publicar en Wallapop/Coches.net automáticamente sin API oficial.

- **Tecnología: Playwright** (Headless Browser).

- **Flujo:**
    1. El sistema inicia sesión con las credenciales del dealer (guardadas cifradas en Vault).
    2. Navega al formulario de "Nuevo Anuncio".
    3. Rellena campos y sube fotos simulando ser un humano (tecleo, clics).
- **Riesgo:** Cambios en el formulario de los portales.
- **Mitigación: Self-Healing Scripts**. Si el selector del botón "Publicar" cambia, la IA (vía análisis del DOM) busca el botón más probable y actualiza el script.

5.4. MARKETPLACE DE SERVICIOS FÍSICOS (GEO-DISPATCHING)

La "Uberización" de los peritos mecánicos.

5.4.1. ARQUITECTURA DE DISPATCHING

- **Base de Datos Geoespacial: PostGIS** (extensión de Postgres).
- **Indexación:** Usamos índices **H3** (Hexágonos) para localizar mecánicos.
- **El Algoritmo de Asignación:**
    1. Usuario pide revisión en Coordenada X,Y.
    2. Query PostGIS: `SELECT * FROM mechanics WHERE ST_DWithin(location, POINT(X,Y), 20000) AND status = 'AVAILABLE'`.
    3. Envío de Notificación Push (Firebase FCM) a los 3 mecánicos más cercanos.
    4. Primer "Aceptar" gana el trabajo (Race Condition gestionada por transacciones de base de datos atómicas).

5.4.2. LA APP DEL MECÁNICO (FLUTTER)

- **Tecnología: Flutter** (Un código para iOS y Android).
- **Funcionalidad Offline:** El mecánico puede rellenar el checklist en un garaje sin cobertura. Los datos se sincronizan (CRDTs - Conflict-free Replicated Data Types) cuando recupera conexión.
- **Coste Desarrollo:** Medio.
- **Coste Operativo:** Firebase (Capa gratuita generosa) + Servidor propio.

5.5. UNIT ECONOMICS: ANÁLISIS DE VIABILIDAD (LA PRUEBA DEL ALGODÓN)

¿Es rentable esto o vamos a quemar dinero?

ESCENARIO: DEALER "PRO" (50 COCHES EN STOCK)

- **Ingresos:** 299€ / mes (Suscripción).
- **Costes Mensuales CARDEX:**
    1. **Almacenamiento (50 coches x 20 fotos x 100KB AVIF):** 100MB. Coste disco NVMe: **0,005€**.
    2. **Marketing IA (50 descripciones x 300 tokens):** Procesado en local. Coste electricidad CPU: **0,02€**.
    3. **Remasterización Fotos (1000 fotos):** Procesado CPU (Rembg). Coste electricidad: **0,15€**.
    4. **Multiposting (Actualización diaria):** Ancho de banda + CPU: **0,50€**.
    5. **Infraestructura Base (Prorrateo Servidor): 1,50€**.
- **COSTE TOTAL SERVICIO: ~2,17€ / mes.**
- **MARGEN BRUTO: 296,83€ (99,2%).**

VEREDICTO: El modelo es una máquina de imprimir dinero si, y solo si, mantenemos el cómputo de IA en local (Hetzner) y no pagamos APIs por cada foto/texto.

MÓDULO 6.0: AGENCIA DE MARKETING AUTOMATIZADA (EL CMO ARTIFICIAL)

CLASIFICACIÓN: GENERACIÓN DE INGRESOS / RETENCIÓN

OBJETIVO: MONETIZAR LA INCOMPETENCIA DIGITAL DEL DEALER.

FILOSOFÍA: "TU TRABAJO ES COMPRAR Y VENDER COCHES. EL NUESTRO ES QUE PAREZCAS UNA MULTINACIONAL."

6.1. MOTOR DE DIAGNÓSTICO: EL AUDITOR FORENSE

Antes de vender la solución, vendemos el problema. El sistema analiza automáticamente la presencia digital de cada dealer detectado en el Módulo 2.

6.1.1. ALGORITMO DE "DIGITAL GAP ANALYSIS"

El sistema escanea al dealer (su perfil de Google, su Facebook, su web si la tiene) y genera un **Score de Competitividad (0-100)** basado en hechos técnicos:

1. **Auditoría Visual (IA Visión):**
    - *Input:* Las últimas 10 fotos subidas.
    - *Análisis:* ¿Iluminación < 30% (Oscuras)? ¿Resolución < 720p? ¿Fondo sucio (basura, otros coches)?
    - *Veredicto:* "Tus fotos devalúan tu stock un 15%".
2. **Auditoría de Respuesta:**
    - *Test:* El bot envía un mensaje genérico ("¿Sigue disponible?") a las 10:00 AM.

    - *Medición:* Tiempo de respuesta. Si > 4 horas -> Penalización grave.
3. **Auditoría SEO/Web:**
    - ¿Tiene web? Si no -> Oportunidad crítica.
    - Si tiene: ¿Carga en <2s? ¿Es responsive? (Lighthouse Score).

6.1.2. EL INFORME "SHOCK" (LEAD MAGNET)

- **Generación:** PDF automático enviado por WhatsApp/Email al dealer.
- **Contenido:** No opiniones, datos. "Tu competencia directa (Dealer X, Dealer Y) vende este Golf en 12 días. Tú tardas 28 días. La diferencia son tus fotos y tu falta de web. Aquí tienes la solución."

6.2. EL ESTUDIO FOTOGRÁFICO NEURONAL (INGENIERÍA DE IMAGEN)

El dealer saca fotos con un móvil barato en un polígono. Nosotros las transformamos en catálogo de Audi. **Sin costes de API (todo local en Hetzner).**

6.2.1. PIPELINE DE PROCESAMIENTO (CPU/GPU HÍBRIDO)

1. **Segmentación Semántica (Rembg / U2Net):**
    - Separa el coche del fondo con precisión de píxel.
    - *Optimización:* Máscara alfa para ventanas (para que se vea el fondo a través de los cristales, realismo puro).
2. **Reconstrucción de Entorno (Compositing):**
    - No usamos IA generativa pura (alucina ruedas o espejos). Usamos **Proyección de Planos**.
    - El coche recortado se proyecta sobre "Escenarios 3D Pre-renderizados" (Un estudio blanco infinito, un paisaje urbano de lujo, un atardecer).
    - **Ajuste de Iluminación (Color Grading):** Script de OpenCV que iguala el histograma de color del coche con la iluminación del fondo virtual.
3. **Anonimización y Branding:**
    - Detección de matrículas (YOLOv8).
    - Reemplazo automático por una placa virtual con el logo del Dealer.
    - Marca de agua sutil.

- **Coste:** 0€ (Cómputo local).
- **Tiempo:** < 3 segundos por foto.

6.3. LA FÁBRICA DE WEBS INSTANTÁNEAS (INFRAESTRUCTURA AS CODE)

El dealer no quiere "hacer una web". Quiere tenerla hecha.

6.3.1. ARQUITECTURA MULTI-TENANT (NEXT.JS)

No desplegamos un WordPress por cliente (eso es inmanejable). Usamos una **Arquitectura de Plataforma Única**.

- **Tecnología:** Next.js con Middleware para subdominios.
- **Funcionamiento:**
  - El dealer se registra como "Autos Pepe".
  - El sistema asigna autospepe.cardex.shop (o su dominio propio).
  - **Base de Datos:** No hay base de datos separada. La web consulta la API de CARDEX filtrando por dealer_id.
- **Despliegue:** Cero tiempos de espera. Al hacer clic en "Activar Web", el dominio resuelve instantáneamente.

6.3.2. SEO PROGRAMÁTICO (DOMINACIÓN DE GOOGLE)

Las webs de dealers normales son invisibles. Las de CARDEX están dopadas.

- **Generación de Landing Pages:** Por cada coche, generamos una URL estática (/bmw-serie-3-madrid-2020) con Schema Markup (JSON-LD) perfecto para que Google indexe el precio, el stock y las reviews.
- **Copywriting IA (Qwen 2.5):** Generamos descripciones únicas para cada coche para evitar contenido duplicado, enfocadas en keywords locales ("Coche segunda mano en [Ciudad]").

6.4. EL PULPO DE DISTRIBUCIÓN (MULTIPOSTING RPA)

Publicar el coche en la web propia no basta. Hay que estar en los marketplaces.

6.4.1. ARQUITECTURA DE "HEADLESS PUBLISHERS"

Las APIs oficiales de publicación (Coches.net, Mobile.de) son caras o solo para grandes cuentas. Nosotros usamos **RPA (Robotic Process Automation)**.

- **Cluster de Navegadores (Playwright):**
  1. Cada portal tiene un "Driver" específico (Publisher_Wallapop, Publisher_Facebook).
  2. **Gestión de Sesión:** Cookies persistentes del dealer.
- **Flujo de Publicación:**
  1. El dealer sube el coche a CARDEX una vez.
  2. El sistema pone en cola 5 tareas de publicación.
  3. Los bots navegan, rellenan formularios, suben las fotos mejoradas y publican.
  4. **Reporte:** El dealer recibe un link a sus 5 anuncios en vivo.

6.4.2. SINCRONIZACIÓN DE ESTADO (WEBHOOKS INVERSOS)

- **Problema:** Si vende el coche en la tienda, el anuncio sigue en Wallapop y le llaman molestando.

- **Solución:** Botón "Vendido" en CARDEX -> **Borrado en cascada**.
    - El sistema lanza los bots para retirar el anuncio de todas las plataformas en <5 minutos.

6.5. NEUROMARKETING CONVERSACIONAL (CHATBOT DE VENTAS)

Cuando llega un lead (mensaje), el dealer tarda horas en contestar. CARDEX contesta en segundos.

6.5.1. INBOX UNIFICADO

- Centralizamos chats de: WhatsApp Business API, Facebook Messenger, Wallapop Chat (vía scraping del chat) y Email.
- Todo llega a una sola pantalla en la App del dealer.

6.5.2. EL AGENTE DE VENTAS (IA LOCAL)

- **Modelo:** Llama-3-8B-Instruct (Fine-tuned para ventas).
- **Función:**
    - Responde preguntas técnicas: *"¿Tiene etiqueta ECO?"* -> Consulta la ficha técnica -> *"Sí, tiene etiqueta ECO y gasta 4.5L/100km"*.
    - Agenda citas: *"¿Te viene bien pasar a verlo mañana a las 10?"*.
    - **Escalado:** Si detecta intención de compra fuerte o pregunta compleja, notifica al dealer: **"CLIENTE CALIENTE EN WHATSAPP AHORA"**.

6.6. UNIT ECONOMICS DEL MÓDULO 6 (RENTABILIDAD)

SERVICIO	COSTE REAL (CARDEX)	PRECIO MERCADO	PRECIO CARDEX	MARGEN
Web Dealer	0,05€/mes (Hosting)	100€/mes	**49€/mes**	**99,9%**
Fotos IA	0,001€/foto (Luz)	2€/foto (Retocador)	**Incluido**	**N/A**

Multiposting	0,10€/anuncio (Proxies)	50€/mes (Gestor)	**29€/mes**	**High**
Chatbot IA	0,00€ (Local)	30€/mes	**Incluido en Pro**	**N/A**

IMPACTO ESTRATÉGICO:

Este módulo convierte a CARDEX en el "oxígeno" del dealer. Si deja de pagar, su web desaparece, sus fotos vuelven a ser feas y tiene que subir los anuncios a mano uno a uno. **Churn Rate = 0%.**

MÓDULO 7.0: MARKETPLACE DE SERVICIOS FÍSICOS ("EL PUENTE AL MUNDO REAL")

CLASIFICACIÓN: OPERACIONES DE CAMPO / MONETIZACIÓN TRANSACCIONAL **OBJETIVO:** ELIMINAR LA DISTANCIA FÍSICA. QUE COMPRAR UN COCHE EN BERLÍN SEA IGUAL QUE COMPRARLO EN LA ESQUINA. **FILOSOFÍA:** "UBERIZACIÓN DEL PERITAJE" Y "ALGORITMIA DE RUTAS VACÍAS".

7.1. RED DE INSPECCIONES IN-SITU (UBER PARA MECÁNICOS)

No contratamos mecánicos. Orquestamos una fuerza de trabajo descentralizada.

7.1.1. ARQUITECTURA DE DISPATCHING (ASIGNACIÓN)

El problema es el tiempo. Un dealer no puede esperar 3 días a una revisión. Necesitamos respuesta en horas.

A. EL ALGORITMO "PROXIMITY-MATCH":

1. **Indexación Geoespacial:** Cada mecánico activo emite un "latido" (heartbeat) de GPS cada 5 minutos. Indexamos su posición en **Redis Geo** o **PostGIS** usando celdas H3 (Resolución 9).
2. **Lógica de Asignación (La Carrera):**
    - El Cliente pide revisión en Coordenada X.
    - El sistema busca mecánicos en radio 20km.

    - **Priorización:** Score de Reputación > Velocidad de Respuesta > Distancia.
    - **Notificación Push (FCM):** "Nueva Misión: Audi A4, 45€. A 15 min de ti. ¿Aceptas?".
    - **Bloqueo:** El primero que acepta bloquea la misión. (Modelo *First-come, first-served* para maximizar velocidad).

7.1.2. LA "APP DE CAMPO" (ANTI-FRAUDE)

El mecánico puede ser perezoso o corrupto (amigo del vendedor). La App no le deja serlo.

TECNOLOGÍA: Flutter (iOS/Android nativo) con lógica "Offline-First" (base de datos local **WatermelonDB**).

PROTOCOLOS DE SEGURIDAD OPERATIVA:

1. **Cámara Blindada:**
    - La App **bloquea la subida de fotos de la galería**. El mecánico *tiene* que hacer la foto en ese momento.
    - **Geotagging Forzoso:** Cada foto lleva metadatos GPS incrustados y firmados criptográficamente. Si la foto se hace a 500m del taller del vendedor -> **ALERTA DE FRAUDE.**
2. **Checklist Secuencial Bloqueante:**
    - No puede saltar pasos.
    - Paso 1: Foto VIN (OCR valida que es el coche correcto).
    - Paso 2: Arrancar motor (Grabar Audio obligatorio 30s -> Análisis de espectro de sonido por IA en servidor para detectar ruidos anómalos).
    - Paso 3: Medidor de espesor de pintura (Foto del valor).
3. **Time-Fencing:**
    - Si la revisión de un "Chequeo Completo" dura menos de 15 minutos -> **Revisión invalidada.** Se exige repetición.

7.1.3. EL INFORME DE SALIDA (VIVO)

- No es un PDF estático al final. Es un **Streaming de Verdad**.
- El cliente recibe las fotos y vídeos en tiempo real conforme el mecánico las sube.
- **Veredicto Final:** El mecánico no dice "Está bien". Marca casillas estandarizadas (Fugas, Humo, Ruido). El **Servidor** calcula el "Score Mecánico" (0-100) para evitar subjetividad.

7.2. LOGÍSTICA DE RETORNO (EL ALGORITMO DE "EMPTY LEGS")

Transportar aire es caro. Transportar metal en huecos vacíos es barato.

No vamos a comprar camiones. Vamos a crear un **Mercado Spot de Cargas**.

7.2.1. ARQUITECTURA DE INTEGRACIÓN

1. **Conexión API Mayorista:**
    - Integración con grandes operadores (Hödlmayr, Gefco) para rutas troncales (ej: Múnich -> Campa Madrid). Precios negociados por volumen anual.
2. **El "Bolsín de Cargas" (Mercado Secundario):**
    - Para rutas capilares (del taller perdido a la campa).
    - Los transportistas autónomos con la App "CARDEX Carrier" ven cargas disponibles en su ruta de vuelta.
    - **Subasta Inversa:** "Tengo un hueco en el camión de Badajoz a Madrid. Lo llevo por 150€ (precio normal 300€)".

7.2.2. TRACKING UNIFICADO (LA TORRE DE CONTROL)

El cliente no quiere saber quién lo lleva. Quiere saber dónde está.

- **Normalización de Eventos:**
    - Traducimos los estados de 50 proveedores (`ARRIVED_HUB`, `ON_TRUCK`, `DELIVERED`) a un estándar CARDEX (`EN_TRANSITO`).
- **Predicción de Llegada (ETA):**
    - Modelo de ML simple (`RandomForest`) que estima la llegada basándose en el histórico de ese transportista en esa ruta específica + Tráfico en tiempo real (Google Routes API).

7.3. INFRAESTRUCTURA TÉCNICA DEL MÓDULO 7

¿Dónde corre esto en el AX102?

- **Servidor de WebSockets (Node.js/Go):**
    - Para la geolocalización en tiempo real de los mecánicos y el estado de la misión.
    - Mantiene conexiones persistentes de bajo consumo.
- **Almacenamiento de Media (S3 Compatible / MinIO):**
    - Los vídeos de sonido de motor y fotos HD consumen mucho.
    - Usaremos **Hetzner Storage Box** (o MinIO local en discos HDD grandes) montado como sistema de archivos. Coste irrisorio por TB.
- **Procesador de Audio (IA):**
    - Contenedor Docker con librería **Librosa** o modelo de audio ligero (YAMNet) para detectar "golpeteo de biela" en el audio subido por el mecánico.

7.4. UNIT ECONOMICS Y MODELO DE PAGO

Cómo ganamos dinero moviendo átomos.

7.4.1. INSPECCIONES

- **Precio al Cliente:** 149€ (Estándar mercado).
- **Pago al Mecánico:** 90€ (Sueldo alto para 1h de trabajo = Alta fidelización).
- **Coste Operativo:** 2€ (Servidor + Transferencia).
- **Margen Neto: 57€ (38%)**.
- **Gestión de Pagos (Stripe Connect):** El dinero se retiene en *Escrow*. Se libera al mecánico solo cuando el informe está subido y validado por la IA (Geolocalización OK).

7.4.2. LOGÍSTICA

- **Modelo:** Comisionista puro.
- **Precio:** Coste del Transportista + 15% Fee de Gestión.
- **Valor para el Cliente:** Seguro de carga incluido y gestión de incidencias centralizada. No tiene que llamar al camionero rumano; habla con CARDEX.

MÓDULO 8.0: INGENIERÍA FINANCIERA Y MODELO DE DOMINACIÓN

CLASIFICACIÓN: ESTRATEGIA DE CAPITAL / TOP SECRET

OBJETIVO: LOGRAR "NEGATIVE CHURN" (QUE EL CLIENTE NOS PAGUE MÁS CADA MES AUTOMÁTICAMENTE).

FILOSOFÍA: "NO VENDEMOS DATOS. VENDEMOS DINERO (ALPHA)."

8.1. ARQUITECTURA DE PRICING (LA PIRÁMIDE DE VALOR)

El error de *Dealcar* o *Auto1* es cobrar por "acceso". Eso es commodity.

Nosotros cobramos por **Velocidad** y **Arbitraje**.

8.1.1. TIER 0: "THE TROJAN HORSE" (GRATIS - LOCK-IN TOTAL)

- **Objetivo:** Matar la barrera de entrada. Captar al dealer analógico.
- **Incluye:**
  - CRM Básico (Gestión de Stock).
  - Web Propia Automatizada (Módulo 6).
  - Publicación en Portales (Manual).
- **El Truco:** Le damos una herramienta que vale 200€/mes gratis.
- **La Trampa:** Sus datos ahora son nuestros. Su stock alimenta nuestro algoritmo de precios. Si se va, pierde su web y su gestión. **Churn Técnico = 0%.**

8.1.2. TIER 1: "HUNTER" (EL SAAS OPERATIVO)

- **Target:** Compraventas pequeños/medianos (Stock < 50 coches).
- **Precio: 299 CHF / mes**.
- **Incluye:**
    - Búsqueda en los 12 Millones de Activos (Módulos 1 y 2).
    - Historial de Vehículo Gratuito (Módulo 4).
    - Alertas de Mercado en Tiempo Real (Push).
- **Economics:** Coste marginal para nosotros: ~2 CHF. **Margen Bruto: 99%.**

8.1.3. TIER 2: "SNIPER" (LA MÁQUINA DE DINERO)

- **Target:** Grandes Traders, Concesionarios Oficiales, Inversores.
- **Precio: 999 CHF / mes + 1% de "Success Fee" sobre Arbitraje (Opcional).**
- **Incluye:**
    - **Algoritmo de Arbitraje Fiscal (Módulo 3):** Ve el margen neto real.
    - **Acceso a Subastas B2B "Sleeper" (Módulo 2.2):** Datos ocultos.
    - **API Access:** Para conectar sus propios ERPs.
    - **Priority Logistics:** Descuento en transporte.

8.2. UNIT ECONOMICS: LA MATEMÁTICA DE LA RENTABILIDAD

Aquí demostramos por qué CARDEX es una máquina de imprimir billetes frente a la competencia que usa AWS/Google Cloud.

ESCENARIO BASE: CLIENTE "TIER 1" (HUNTER - 299 CHF)

CONCEPTO (COSTE MENSUAL)	COMPETENCIA (AWS/AZURE)	CARDEX (HETZNER BARE METAL)	FACTOR DE VENTAJA
Cómputo (Scraping)	45 CHF (EC2 Instances)	2.50 CHF (AX102 Prorrateado)	**18x**
Almacenamiento (Fotos)	20 CHF (S3 API Calls)	0.50 CHF (NVMe Propio)	**40x**
IA (Visión/NLP)	30 CHF (OpenAI/Google API)	1.50 CHF (Modelos Locales)	**20x**

Proxies (Residenciales)	50 CHF (Sin optimizar)	15 CHF (Routing Híbrido 1.3)	**3.3x**
COSTE TOTAL SERVICIO	**~145 CHF**	**~19.50 CHF**	**7.4x**
MARGEN DE CONTRIBUCIÓN	**154 CHF (51%)**	**279.50 CHF (93.4%)**	**IMBATIBLE**

CONCLUSIÓN: Podemos bajar el precio a la mitad y seguir ganando más dinero que ellos. Tenemos "Foso Económico" (Economic Moat) estructural.

8.3. FLYWHEEL DE LIQUIDEZ (MODELO DE MARKETPLACE)

El SaaS (Suscripción) paga las facturas. El Marketplace (Comisiones) paga el yate.

8.3.1. TAKE RATE (LA TASA DE MORDIDA)

Cada vez que un dealer usa CARDEX para algo físico, cobramos.

- **Inspecciones Mecánicas:**
  - Precio Cliente: 149 CHF. Coste Mecánico: 90 CHF.
  - **Net Revenue: 59 CHF (Margin 40%)**.
- **Transporte (Logística):**
  - Precio Cliente: 600 CHF. Coste Transportista: 500 CHF.
  - **Net Revenue: 100 CHF (Margin 16%)**.
- **Financiación / Garantía:**
  - Referral Fee de bancos/aseguradoras: **150 - 300 CHF por coche**.

8.3.2. EL CÍRCULO VIRTUOSO (NETWORK EFFECT)

1. Damos el SaaS barato -> **Más Dealers entran.**
2. Más Dealers -> **Más datos de stock real y transacciones.**
3. Más datos -> **Algoritmo de Precios (Módulo 3) más preciso.**
4. Precios precisos -> **Menor riesgo para el Dealer.**
5. Menor riesgo -> **Más Dealers entran.**

8.4. ESTRATEGIA DE LANZAMIENTO "BLITZSCALING" (GUERRA RELÁMPAGO)

No vamos a llamar puerta a puerta. Vamos a hackear el crecimiento.

8.4.1. FASE 1: LA INYECCIÓN DE DATOS (MES 1-3)

- **Acción:** Scrapeamos el 100% de los dealers de un país (ej: España).
- **Ataque:** Generamos 5.000 webs automáticas con su stock real (Módulo 6).
- **Cold Outreach:** Bot de WhatsApp masivo: *"Hola Taller Pepe, he visto que no tienes web. Aquí tienes la tuya hecha: [link]. Es gratis. Si quieres editar el precio de este BMW, entra aquí."*
- **Conversión Estimada:** 5-10% (Muy alta porque el valor ya está entregado).

8.4.2. FASE 2: EL CANDADO (LOCK-IN)

- Una vez dentro, el dealer recibe una alerta: *"Un cliente busca un coche como el tuyo en Múnich. Véndeselo."*
- Para ver el contacto -> **Upgrade a TIER 1**.

8.5. MONETIZACIÓN DE "DATA EXHAUST" (EL ACTIVO OCULTO)

Los bancos y aseguradoras pagan millones por saber el valor real de los coches.

- **Producto API para Bancos:** "Cardex Valuation API".
- **Uso:** El banco la usa para decidir si da un préstamo de 20.000€ por un coche.
- **Precio: 0,10 CHF por llamada API**.
- **Volumen:** Millones de llamadas al mes. Ingreso pasivo puro con margen del 100%.

8.6. PROYECCIÓN FINANCIERA DE ALTO NIVEL (EL OBJETIVO)

Meta a 24 Meses (Conservadora)

- **Usuarios Activos (Dealers):** 5.000 (2% del mercado EU).
- **Ratio de Conversión a Pago:** 20% (1.000 clientes de pago).
- **MRR (SaaS):** 1.000 * 299 CHF = **299.000 CHF / mes**.
- **GMV Marketplace (Servicios):** 500 inspecciones/mes + 200 transportes.
    - Revenue Marketplace: ~50.000 CHF / mes.
- **Coste Infraestructura (Hetzner):** ~3.000 CHF / mes.
- **EBITDA MENSUAL: ~346.000 CHF**.
- **VALORACIÓN DE EMPRESA (MULTIPLO SAAS 10x ARR): ~40 MILLONES CHF.**

MÓDULO 9.0: INGENIERÍA LEGAL,

COMPLIANCE Y DEFENSA CORPORATIVA

CLASIFICACIÓN: DEFENSA ESTRATÉGICA / NIVEL DE RIESGO: CRÍTICO
JURISDICCIÓN: SUIZA (KANTON ZUG/ZÜRICH) **OBJETIVO:** BLINDAJE TOTAL CONTRA DEMANDAS POR PROPIEDAD INTELECTUAL (IP), RGPD Y COMPETENCIA DESLEAL.
FILOSOFÍA: "NO SOMOS PIRATAS. SOMOS UN MOTOR DE BÚSQUEDA ESPECIALIZADO."

9.1. ESTRUCTURA CORPORATIVA "AIRLOCK" (CORTAFUEGOS DE RESPONSABILIDAD)

El error de novato es poner el scraper, la base de datos y la facturación en la misma SL/AG. Si demandan a una, cae todo.

LA TRÍADA DE DEFENSA (ESTRUCTURA HOLDING):

1. **CARDEX OPS AG (La Operadora - "La Carne de Cañón"):**
    - **Función:** Ejecuta los scrapers, firma los contratos de proxies, "toca" la web de Mobile.de.
    - **Activos:** CERO. Solo servidores alquilados y saldo operativo para 1 mes.
    - **Riesgo:** Si Mobile.de demanda por violación de Términos de Servicio (ToS), esta empresa absorbe el golpe. Si quiebra, abrimos "Cardex Ops II AG" al día siguiente.
2. **CARDEX IP HOLDING (La Dueña - "La Caja Fuerte"):**
    - **Función:** Dueña del Código Fuente, la Marca y los Algoritmos (Módulo 3). Licencia el software a *Ops AG*.
    - **Activos:** TODO.
    - **Blindaje:** Nadie puede tocar el código porque esta empresa *no scrapea*. Solo "alquila tecnología". Es intocable legalmente por acciones de terceros.
3. **CARDEX SALES (La Cara Visible):**
    - **Función:** Factura al cliente final (SaaS).
    - **Defensa:** "Nosotros solo compramos datos a proveedores externos (Ops AG)". Desconocimiento de la procedencia ilícita del dato.

9.2. PROTOCOLO DE DEFENSA RGPD / nFADP (LA TRITURADORA DE DATOS)

Estamos en Suiza (Nueva Ley de Protección de Datos - nFADP), pero tratamos datos de ciudadanos de la UE (RGPD). Las multas son del 4% de la facturación global.

EL ARGUMENTO LEGAL (ART. 6.1.F RGPD - INTERÉS LEGÍTIMO): No pedimos consentimiento para scrapear. Nos basamos en el **Interés Legítimo**.

- *Tesis:* "CARDEX indexa información pública para garantizar la transparencia del mercado y prevenir el fraude en el kilometraje."
- *Requisito:* El beneficio para la sociedad (evitar estafas) supera el perjuicio a la privacidad del vendedor.

LA ARQUITECTURA "TOXIC WASTE" (GESTIÓN DE DATOS PERSONALES): El nombre, teléfono y foto del vendedor particular son "Residuos Tóxicos".

1. **Segregación B2B/B2C:**
    - Si el vendedor es **Empresa** (CIF/VAT detectado, más de 3 coches en venta) -> Datos Públicos Empresariales. **Scraping Permitido.**
    - Si el vendedor es **Particular** -> **Zona Roja.**
2. **Protocolo de Retención Dinámica:**
    - Mientras el coche está en venta -> Guardamos datos (Finalidad activa).
    - En el momento que detectamos "Vendido" -> **KILL SWITCH AUTOMÁTICO.**
        - Se borra el Nombre y Teléfono de la DB "Live".
        - Se mantiene el VIN y KMs (Datos del coche, no de la persona).
    - **Resultado:** Si la Agencia de Protección de Datos nos audita, nuestra base de datos está "limpia". Solo tenemos datos de coches, no de personas (salvo las activas).

9.3. DEFENSA CONTRA "CEASE & DESIST" (EVASIÓN DE TÉRMINOS DE USO)

Mobile.de enviará cartas de abogados alegando que violamos sus Términos y Condiciones (ToS) que prohíben bots.

LA DEFENSA TÉCNICA (NULLIFICATION OF CONTRACT): Para que un contrato (ToS) sea válido, debes aceptarlo.

1. **Evasión de "Clickwrap":**
    - Nuestros bots están programados a nivel de protocolo (Módulo 1.3) para **NUNCA** cargar el archivo `/terms` ni hacer clic en "Acepto Cookies" o "Acepto Condiciones".
    - Desde el punto de vista forense digital, nuestro "usuario" nunca vio el contrato.
2. **Doctrina "HiQ Labs vs LinkedIn" (Precedente US/EU):**
    - La información pública en internet es... pública. Scrapear datos abiertos *sin login* no es hacking (CFAA).
    - **Línea Roja:** Jamás scrapearemos datos detrás de un login *privado* (Facebook Friends) usando fuerza bruta. Scrapeamos lo que Google puede ver.

9.4. PROPIEDAD INTELECTUAL: DERECHO SUI GENERIS (BASES DE DATOS)

En Europa, existe el "Derecho sobre Bases de Datos" (Directiva 96/9/CE). Si copias la base de datos de AutoScout, te demandan.

LA ESTRATEGIA DE "TRANSFORMACIÓN SUSTANCIAL": No copiamos. **Creamos una obra nueva.**

- **Input:** Datos sucios de Mobile.de.
- **Proceso (Módulo 2.3):**
  - Normalización vectorial.
  - Corrección de equipamiento vía VIN.
  - Traducción y enriquecimiento IA.
- **Output (CARDEX DB):** El resultado final es tan distinto del original que constituye una "Nueva Base de Datos" protegida por *nuestro* propio derecho de autor.
- **Defensa:** "Señoría, nosotros no copiamos Mobile.de. Usamos Mobile.de como *materia prima* (fcts) para generar nuestra propia inteligencia."

9.5. FISCALIDAD Y VAT (INGENIERÍA TRIBUTARIA)

Vendemos servicios digitales desde Suiza a la UE.

EL MECANISMO "REVERSE CHARGE":

- Facturamos desde **CARDEX SALES AG (Suiza)**.
- Al cliente B2B (Dealer en España/Alemania): Factura **SIN IVA** (Inversión del Sujeto Pasivo).
  - *Ventaja Psicológica:* El dealer ve "299€" y paga "299€". No adelantamos IVA a Hacienda. Flujo de caja optimizado.
- Al cliente particular (Si lo hubiera): Nos registramos en la ventanilla única (OSS) de la UE, pero nuestro foco es 99% B2B para evitar este dolor de cabeza.

9.6. INFRAESTRUCTURA DE CUMPLIMIENTO TÉCNICO

Cómo el código cumple la ley automáticamente.

A. EL BOTÓN "DERECHO AL OLVIDO" AUTOMATIZADO:

- Por ley (RGPD Art. 17), cualquiera puede pedir que borres sus datos.
- **Implementación:** Formulario web `/privacy-request`.

    - Input: Teléfono o Email.
    - Acción: El sistema busca en todas las tablas (SQL + ClickHouse) y ejecuta un `HARD DELETE` instantáneo y añade el hash del teléfono a una **"Blacklist de No-Scrapear"**.
    - **Efecto:** Si volvemos a encontrar a ese vendedor, el scraper lo ignora en el origen. Evitamos reincidencia y multas.

B. REGISTRO DE ACTIVIDADES DE TRATAMIENTO (RAT):

- Script automático que genera un log inmutable de:
    - Qué datos scrapeamos.
    - Para qué (Finalidad: Valoración de mercado).
    - Cuándo se borran (TTL).
- Esto es lo primero que pide un auditor. Tenerlo automatizado demuestra "Responsabilidad Proactiva" (Accountability).

MÓDULO 10.0: PROTOCOLO "HYDRA" (RESILIENCIA EXISTENCIAL Y GUERRA ASIMÉTRICA)

CLASIFICACIÓN: BLACK OPS / NIVEL DE AMENAZA: EXISTENCIAL **OBJETIVO:** QUE EL COSTE DE BLOQUEARNOS SEA SUPERIOR AL COSTE DE IGNORARNOS. **FILOSOFÍA:** "CORTA UNA CABEZA, SALEN TRES. NO SOMOS UN SERVIDOR, SOMOS UN VIRUS."

10.1. INFRAESTRUCTURA POLIMÓRFICA (MOVING TARGET DEFENSE)

El error es tener IPs estáticas. Mobile.de apunta y dispara. Nosotros seremos **Líquido**.

10.1.1. ARQUITECTURA "SERVERLESS GHOST"

- **El Concepto:** No usamos servidores persistentes para el scraping crítico.
- **La Ejecución:**
    - Desplegamos **Funciones Efímeras (AWS Lambda / Google Cloud Run / Fly.io)** en 15 regiones simultáneas.
    - Cada función nace, scrapea 10 coches y **se autodestruye** en 30 segundos.
    - **Resultado:** Mobile.de no puede bloquear una IP que ya no existe. Su firewall dispara al aire.

10.1.2. ENRUTAMIENTO VÍA "RESIDENTIAL IOT BOTNET" (ZONA GRIS)

Olvídate de Bright Data (es caro y detectado). Vamos al metal.

- **Acuerdo con Proveedores SDK:** Integramos nuestro SDK de proxy en Apps gratuitas de Android (Linternas, Calculadoras) a cambio de monetización para el desarrollador.
- **Efecto:** Nuestro tráfico sale desde el móvil de "Pepito en Múnich" y "María en Hamburgo".
- **Imbloqueable:** Para Mobile.de, bloquearnos significa bloquear a sus propios usuarios legítimos. **Jaque Mate Técnico.**

10.2. CONTRA-INTELIGENCIA PREDICTIVA (SABER QUE VIENEN)

No esperamos al bloqueo. Lo anticipamos.

10.2.1. MONITORIZACIÓN DE "CANARIES" (ESPIONAJE TÉCNICO)

- **El Señuelo:** Colocamos 50 "Coches Trampa" (anuncios falsos nuestros) en Mobile.de.
- **El Análisis:** Monitorizamos quién visita esos anuncios.
    - Si vemos visitas desde rangos de IP de **Google (Bot)** -> Normal.
    - Si vemos visitas desde rangos de **Hetzner/AWS** con User-Agent headless -> **Es el Bot de Seguridad de Mobile.de escaneando.**
- **Acción:** Si detectamos su scanner, nuestro sistema le sirve contenido "limpio" (Cloaking) para que crea que somos usuarios normales.

10.2.2. INFILTRACIÓN EN COMUNIDADES DEV

- Scraper activo en **StackOverflow, Reddit y GitHub** buscando keywords: *"Mobile.de anti-bot", "Akamai update", "Datadome patch"*.
- Si un ingeniero de Mobile.de pregunta sobre una nueva técnica de bloqueo, el sistema nos alerta **semanas antes** de que la implementen.

10.3. EVASIÓN BIOMÉTRICA (EL "HUMANIZADOR")

Los WAFs modernos (Datadome) no miran la IP, miran cómo mueves el ratón.

10.3.1. GRABACIÓN DE SESIONES REALES (GHOST REPLAY)

- **La Fuente:** En nuestra Web Gratuita para dealers (Módulo 6), grabamos (con su permiso en T&C) exactamente cómo mueven el ratón y teclean.

- **El Payload:** Cuando nuestro bot ataca Mobile.de, no usa un algoritmo aleatorio. **Reproduce** la grabación de una sesión humana real (con sus dudas, sus temblores y sus clics erráticos).
- **Resultado:** Pasamos el test de Turing biométrico.

10.3.2. RESOLUCIÓN DE CAPTCHAS VÍA "CLICKJACKING" (SOCIAL ENGINEERING)

- Si nos salta un Captcha imposible.
- **Redirección:** Se lo mostramos a un usuario nuestro en la Web Gratuita para acceder a una funcionalidad premium.
- **Efecto:** El usuario resuelve el Captcha de Mobile.de pensando que es seguridad de CARDEX. Nosotros inyectamos el token en nuestro bot. **Coste: 0€. Velocidad: Humana.**

10.4. LA OPCIÓN NUCLEAR (DISUASIÓN LEGAL AGRESIVA)

Si nos envían un Cese y Desista, no respondemos con miedo. Respondemos con fuego.

10.4.1. DEMANDA PREVENTIVA ANTIMONOPOLIO (ANTITRUST LAWFARE)

- Tenemos preparado un dossier legal en Bruselas/Suiza.
- **Argumento:** "Mobile.de tiene una posición dominante y bloquear a CARDEX impide la libre competencia de precios en la UE".
- **Táctica:** En el momento que nos bloquean, presentamos una denuncia ante la **Comisión Europea de Competencia**.
- **Objetivo:** Congelar sus acciones legales contra nosotros mientras se defienden de una multa millonaria de la UE.

10.5. PLAN DE RECUPERACIÓN DE DESASTRES (RTO < 60 SEGUNDOS)

Si Hetzner nos apaga los servidores por abuso (pasa mucho).

10.5.1. LA "MALETA NUCLEAR" (AIR-GAPPED BACKUP)

- **Infraestructura Fantasma:** Tenemos cuentas latentes en **Moldavia (FranTech)** y **Panamá (Offshore Racks)**. Servidores pagados pero apagados.

- **Sincronización:** La base de datos se replica en tiempo real (log shipping) a estos destinos.
- **El Botón Rojo:**
    - Script: `./protocol_lazarus.sh`
    - Acción:
        1. Cambia los DNS de Cloudflare para apuntar a Moldavia.
        2. Levanta los contenedores Docker en el nuevo host.
        3. Restaura el servicio.
    - **Tiempo:** 58 segundos. El cliente ni se entera.

EXTRAS

CARDEX TRADING TERMINAL (CTV) - CORE INFRASTRUCTURE

CLASIFICACIÓN: INFRAESTRUCTURA DE VISUALIZACIÓN DE MERCADO **OBJETIVO:** RENDERIZAR ACTIVOS FÍSICOS (COCHES) COMO SERIES TEMPORALES FINANCIERAS (VELAS). **LATENCIA OBJETIVO:** < 50ms (TIEMPO REAL).

1. INGENIERÍA DEL DATO: CONVERSIÓN "RAW TO OHLC"

El corazón del sistema. Cómo transformamos "anuncios sucios" en "velas japonesas" matemáticas.

A. EL ACTIVO SINTÉTICO (TICKER) Creamos un ID único para cada vehículo tradeable:

- Formato: `MARCA.MODELO.GEN.MOTOR.AÑO`
- Ejemplo: `BMW.M4.G82.COMP.2023`

B. EL ALGORITMO DE AGREGACIÓN (CLICKHOUSE) No guardamos precios estáticos. Calculamos la vela en tiempo de ejecución.

- **OPEN (O):** Precio medio ponderado del Ticker a las 00:00:00 UTC.
- **HIGH (H):** Precio máximo listado en el periodo (Filtrado de outliers > 99th percentil).
- **LOW (L):** Precio mínimo listado (Filtrado de chatarra/siniestros < 5th percentil).
- **CLOSE (C):** Precio medio ponderado del Ticker a las 23:59:59 UTC.
- **VOLUME (V):** Cantidad de unidades únicas (VINs) disponibles en ese periodo.

---

2. ARQUITECTURA DEL SISTEMA (FULL STACK DE BAJA LATENCIA)

A. CAPA DE HIERRO (INFRAESTRUCTURA)

- **Servidor:** Hetzner AX102 (Dedicado).
- **CPU:** AMD Ryzen 9 7950X3D (Para cálculos single-core brutales de agregación).
- **Almacenamiento:** NVMe Gen4 RAID 0 (Velocidad de lectura 14.000 MB/s). *Nota: RAID 0 para velocidad pura, backup externo.*
- **Red:** 1 Gbit/s Dedicado (Uplink).

B. CAPA DE DATOS (TIME SERIES DB)

- **Motor: ClickHouse**.
- **Tabla Principal:** `market_snapshots_mergetree`.
    - Particionada por: `YYYYMM`.
    - Ordenada por: `(TickerID, Timestamp)`.
    - *Razón:* Permite extraer 1 año de datos de un coche en 4 milisegundos.

C. CAPA DE TRANSPORTE (API & SOCKETS)

- **Lenguaje: Go (Golang)**.
- **Protocolo: WebSockets (WSS)** sobre TCP.
- **Framework: Fiber** (Zero allocation).
- **Función:** Mantener un túnel abierto con el navegador del cliente. Cuando ClickHouse detecta un nuevo dato, Go lo "empuja" (Push) al cliente. No hay "Polling" (el cliente no pregunta, el servidor informa).

D. CAPA VISUAL (FRONTEND DE ALTO RENDIMIENTO)

- **Motor de Render: Canvas API** (Aceleración por GPU).
- **Librería Base: TradingView Lightweight Charts** (Versión Standalone).
    - *Configuración:* Fondo `#000000`, Velas `Solid`, Escala Logarítmica opcional.
- **Gestión de Estado: React + Zustand**. (Evitamos re-renderizados de HTML. El Canvas es una caja negra que se pinta sola).

---

3. ESPECIFICACIÓN DE LA INTERFAZ (EL "CANVAS")

El usuario no ve una web. Ve una **Consola de Mando**.

1. **Main Chart Area (80% Pantalla):**
    - Gráfico de Velas (Candlesticks) interactivo.
    - Zoom semántico (De "Año" a "Hora" con scroll del ratón).
    - Crosshair (Cruz de precisión) que muestra Precio y Fecha exactos al pasar el ratón.
2. **Order Book Visual (Derecha):**
    - Histograma horizontal de profundidad. Muestra cuántos coches hay a cada nivel de precio *ahora mismo*.

3. **Ticker Tape (Superior):**
    - Cinta corredera con los activos más volátiles del momento.
4. **Timeframe Selector:**
    - Botones físicos en UI: `1H 4H 1D 1W 1M`.

---

4. COSTES DE DESPLIEGUE (CORE TERMINAL)

CAPEX (Desarrollo Inicial - Estimación Senior Dev):

- **Ingeniería de Datos (ClickHouse Schema + ETLs):** ~80 horas.
- **Backend Go (WebSocket Server):** ~60 horas.
- **Frontend (Integración Canvas + UI Oscura):** ~100 horas.
- **Total Horas:** ~240h.
- **Coste Est. (In-House/Freelance Top): ~12.000€ - 15.000€**.

OPEX (Mensual - Infraestructura):

- **Servidor AX102:** 120€/mes.
- **Licencias Software:** 0€ (Todo Open Source: ClickHouse, Go, TV Library).
- **Mantenimiento:** 0€ (Automático si se configura bien Docker).

---

5. CÓMO SE MONTA (PASO A PASO)

1. **Deploy ClickHouse:** Instalar en el AX102. Configurar la tabla `mergetree` para ingesta masiva.
2. **ETL Pipeline:** Conectar el Módulo 2 (Scrapers) para que vuelquen los JSONs directamente al puerto de ClickHouse.
3. **Go Server:** Levantar el binario en el puerto 8080. Abrir canal WebSocket.
4. **Frontend Build:** Compilar la app Next.js con la librería de gráficos. Conectar al WebSocket `wss://api.cardex.com/stream`.
5. **Test de Estrés:** Meter 10 millones de filas y hacer zoom en el gráfico. Si tarda >100ms, optimizar índices.

ESTO ES EL CHASIS. Sobre esto montaremos después los submodelos (indicadores), pero esto por sí solo ya permite ver el mercado de coches como si fuera el Nasdaq.

ORACLE — INSTITUTIONAL ALPHA CLONING SYSTEM

CLASIFICACIÓN: INTELIGENCIA DE MERCADO / ESPIONAJE ACTIVO

LATENCIA OBJETIVO: < 60 SEGUNDOS (DESDE EL CAMBIO EN ORIGEN HASTA TU PANTALLA)

INFRAESTRUCTURA: HETZNER AX102 (NODO "EYE OF GOD")

1. PRINCIPIO FUNDAMENTAL: "ASYMMETRIC INFORMATION ARBITRAGE"

El mercado es ineficiente. El 99% de los compraventas pierden dinero o ganan miserias. El 1% (Las Ballenas) acumula el 80% del beneficio.

ORACLE no analiza el mercado. ORACLE analiza a los ganadores.

Si el "Dealer X" (con un ROI histórico del 40%) compra un modelo, **es una señal de compra validada matemáticamente**. No necesitamos saber *por qué* lo compró. Solo necesitamos comprarlo nosotros antes que el resto.

---

2. EL NÚCLEO MATEMÁTICO: ALGORITMO DE SELECCIÓN "PREDATOR"

Antes de espiar, debemos saber a quién mirar. No buscamos volumen, buscamos **ALPHA** (Rentabilidad por encima del mercado).

2.1. LA ECUACIÓN DE SCORING (EJECUCIÓN DIARIA EN CLICKHOUSE)

Se ejecuta cada noche sobre la base de datos histórica (12M+ registros).

$$\text{AlphaScore} = (W_1 \cdot \text{VR}) + (W_2 \cdot \text{ZR}) + (W_3 \cdot \text{MK}) - (W_4 \cdot \text{VO})$$

- **VR (Velocity Ratio):** $\frac{\text{Media Mercado DTS}}{\text{Media Dealer DTS}}$. (Si Mercado=40 días y Dealer=10 días $\rightarrow$ VR=4.0).
- **ZR (Zero-Drop Rate):** % de coches vendidos sin NINGUNA bajada de precio. (Indica precisión quirúrgica en la tasación).
- **MK (Markup Spread):** Diferencial entre su precio de venta y el precio medio de compra estimado.
- **VO (Volatility Penalty):** Desviación estándar de sus tiempos de venta. (Penaliza la suerte, premia la consistencia).

FILTRO DE SALIDA:

Solo los **Top 100 Dealers** de Europa con AlphaScore > 90.0 entran en la "Kill List".

---

3. ARQUITECTURA DE VIGILANCIA: "THE SHADOW NET"

Una vez identificados los objetivos, desplegamos una red de vigilancia dedicada. No usamos el scraper general. Usamos **CÉLULAS DE ESPIONAJE**.

3.1. STACK TECNOLÓGICO DE LOS AGENTES (LANGGRAPH + ANTIGRAVITY)

Usamos **LangGraph** para orquestar agentes persistentes con memoria, y **Antigravity** para la evasión física.

- **Agente "SHADOW_STALKER" (Instancia por Dealer):**
  - **Misión:** Monitorizar UN solo perfil de vendedor (ej: autohaus-mueller.de) cada 60 segundos.
  - **Evasión (Antigravity):**
    - **Browser Fingerprint:** Clonamos la huella digital de un usuario local real (User-Agent, Canvas Hash, WebGL Vendor) de la misma ciudad del dealer.
    - **Network:** Proxy Residencial Estático (ISP Grade) geolocalizado. Si el dealer está en Múnich, la IP es de Múnich.
    - **Comportamiento:** El bot simula movimientos de ratón (curvas de Bezier) y scroll aleatorio para engañar a los sistemas anti-bot de Mobile.de/AutoScout.

3.2. FLUJO DE DATOS (PIPELINE)

1. **Extracción:** SHADOW_STALKER descarga el HTML del inventario del dealer.
2. **Hashing:** Genera un Hash SHA-256 de cada coche (ID + Precio + Estado).
3. **Comparación (Redis):** Compara Hash_Actual vs Hash_Anterior (almacenado en RAM).
   - Si $H_{t} \neq H_{t-1} \rightarrow$ **EVENTO DETECTADO**.

---

4. EL MOTOR DE EVENTOS: "SIGNAL PROCESSING CORE"

Identificamos 4 tipos de señales críticas que se envían al usuario.

TIPO A: "THE SNIPER ENTRY" (NUEVO LISTADO)

- **Evento:** El Top Dealer acaba de publicar un coche.
- **Lógica:** Si él lo ha comprado, es bueno.
- **Acción del Sistema:**
  1. Extrae el VIN y especificaciones.
  2. Busca automáticamente en toda Europa (Módulo 2) coches idénticos pero más baratos.
  3. **Alerta al Usuario:** "Dealer Top #3 ha listado BMW M4 a 80k. Encontrados 2 clones en Italia a 72k. MARGEN POTENCIAL: 8k".

TIPO B: "THE PANIC DROP" (BAJADA DE PRECIO)

- **Evento:** El Top Dealer baja el precio > 2%.
- **Lógica:** Se ha equivocado o necesita liquidez urgente.
- **Alerta:** "Señal de Debilidad en Dealer Top #3. Evitar comprar BMW M4, el mercado está cayendo".

TIPO C: "THE INSTANT KILL" (VENTA RÁPIDA)

- **Evento:** Un coche desaparece (vendido) en < 48 horas.
- **Lógica:** Validación absoluta de demanda. Ese modelo es "Líquido Caliente".
- **Acción:** Sube el Score de ese modelo en el sistema general.

---

5. INTERFAZ DE USUARIO: "GHOST PORTFOLIO DASHBOARD"

El usuario no ve "datos". Ve una **Cartera Espejo**.

5.1. COMPONENTES VISUALES (REACT + CANVAS)

1. **The Leaderboard (Tabla Viva):**
    - Lista de los Top 10 Dealers anónimos (ej: "ALPHA_DEALER_01").
    - Métricas en tiempo real: ROI Mensual, Coches Vendidos Hoy, Tiempo Medio Venta.
2. **Live Feed (Cinta de Operaciones):**
    - *"Hace 23s: ALPHA_01 compró Porsche 911 GT3 (992)."*
    - Botón de Acción Inmediata: **[CLONE STRATEGY]**.
3. **Ghost Holdings:**
    - Ves el inventario del Dealer Top como si fuera tuyo.
    - Indicador de "Spread de Imitación": Te dice cuánto ganarías si replicaras su inventario comprando en fuentes baratas.

---

6. IMPLEMENTACIÓN TÉCNICA Y COSTES (REALITY CHECK)

A. INFRAESTRUCTURA (EN TU AX102)

- **Base de Datos:** ClickHouse (Almacenamiento histórico de señales) + Redis (Comparación de Hashes en tiempo real).
- **Cómputo:** 10 Contenedores Docker (uno por cada Dealer Top) corriendo los agentes Python/LangGraph. Consumo de CPU: ~15%.

B. PROXIES (GASTO CRÍTICO)

No podemos usar IPs de Data Center (AWS/Google) porque nos bloquearían al segundo.

- **Proveedor:** Bright Data o Oxylabs (Residential Static IPs).
- **Necesidad:** 10 IPs dedicadas de alta calidad.

- **Coste:** ~150€/mes solo para este módulo. (Es el precio de la invisibilidad).

C. DESARROLLO (HOJA DE RUTA)

1. **Semana 1:** Script SQL en ClickHouse para calcular el Ranking y encontrar a los Top 10.
2. **Semana 2:** Desarrollo del Agente SHADOW_STALKER en Python con nodriver (Antigravity) para evasión.
3. **Semana 3:** Sistema de Hashing y comparación en Redis.
4. **Semana 4:** Frontend (Dashboard) y conexión WebSocket.

---

7. JSON PAYLOAD (EJEMPLO DE ALERTA AL SISTEMA)

Esto es lo que el backend envía al frontend cuando detecta una oportunidad.

JSON


```
{
  "type": "ALPHA_SIGNAL",
  "priority": "CRITICAL",
  "timestamp": "2026-02-11T14:30:05Z",
  "source_dealer": {
    "id": "DE_9921",
    "alias": "WHALE_04",
    "alpha_score": 98.4
  },
  "event": {
    "type": "NEW_LISTING",
    "asset": "AUDI RS6 AVANT PERFORMANCE",
    "listed_price": 145000,
    "currency": "EUR"
  },
  "tactical_analysis": {
```

"message": "Whale_04 entra en RS6. Señal fuerte de demanda.",

"replication_opportunity": [

{

"source": "AUTOSCOUT_ES",

"url": "https://...",

"price": 128000,

"arbitrage_potential": 17000

}

]

}

}

THE SCANNER — HIGH-FREQUENCY OPPORTUNITY RADAR

CLASIFICACIÓN: SISTEMA DE ALERTA TEMPRANA (EWS)

ARQUITECTURA: EVENT-DRIVEN (REDIS PUB/SUB + WEBSOCKETS)

LATENCIA CRÍTICA: < 50ms (DEL NÚCLEO AL PIXEL)

ANCHO DE BANDA: OPTIMIZADO VÍA PROTOCOL BUFFERS (PROTOBUF)

1. LA FILOSOFÍA: "PUSH, DON'T PULL"

El usuario no busca. El sistema **VOMITA** oportunidades.

El Scanner es una columna lateral fija (Sidebar) de 350px que procesa flujos de datos masivos y solo renderiza lo que es matemáticamente rentable.

---

2. LOS "CANALES" MATEMÁTICOS (FILTROS DE GRADO MILITAR)

El radar no muestra ruido. Tiene 4 Motores de Filtrado corriendo en paralelo en el Backend (Go).

CANAL A: ARBITRAGE KINGS (EL ALGORITMO DE MARGEN)

Este canal no mira precios. Mira **DIFERENCIALES**.

Se ejecuta en tiempo real comparando pares de mercados (ej: DE vs ES, DE vs IT, FR vs ES).

La Ecuación de Filtrado (The Spread Logic):

$$\text{NetSpread} = (P_{\text{Target}} \cdot (1 - \text{VAT}_{\text{Dest}})) - (P_{\text{Source}} + \text{Logistics} + \text{RegFees})$$

- **Regla de Ejecución:** Si $\text{NetSpread} > 2.500€$ Y $\text{ROI} > 12\%$, el activo se empuja al Socket.
- **Visualización (Row Density):**
  - **Asset:** AUDI RS3 (8Y)
  - **Delta:** +4.200€ (Verde Neón - #39FF14).
  - **Ruta:** DE -> ES.

CANAL B: DISTRESS & PANIC (VOLATILIDAD NEGATIVA)

Detecta "Sangre". Vendedores que bajan el precio por desesperación o error.

La Ecuación de Detección (The Drop Logic):

$$\Delta P_{\% 1h} = \frac{P_{t} - P_{t-1h}}{P_{t-1h}} \times 100$$

- **Trigger:** Si $\Delta P_{\% 1h} < -5\%$ (Caída brusca en menos de una hora).
- **Contexto:** El sistema cruza este dato con el "Days on Market" (DOM).
  - Si DOM > 90 días + Caída > 10% = **LIQUIDACIÓN TOTAL (COMPRA SEGURA)**.
- **Visualización:**
  - **Asset:** PORSCHE MACAN S
  - **Action:** PANIC DROP (Rojo Sangre - #FF0000).
  - **Value:** -6.500€ (Hoy).

CANAL C: VELOCITY VORTEX (ABSORCIÓN)

Detecta activos que desaparecen antes de que puedas pestañear.

Utiliza el cálculo del **Módulo de Absorción** validado anteriormente.

La Ecuación de Velocidad:

$$V_{\text{score}} = \frac{\text{Unidades Vendidas}_{24h}}{\text{Stock Total Disponible}}$$

- **Trigger:** Si $V_{\text{score}} > 0.4$ (El 40% del stock rota cada día).
- **Uso Táctico:** No importa el precio. Si entra uno, **COMPRA CIEGA**. Se venderá solo.
- **Visualización:**
  - **Asset:** TOYOTA GR86
  - **Status:** CRITICAL STOCK (Naranja Alerta - #FFAE00).
  - **Metric:** 12 Left / 40 Sold.

CANAL D: MEMPOOL SNIPER (TIEMPO CERO)

Conexión directa con el Módulo 1 (Ingesta).

Muestra activos que han sido indexados hace menos de **60 segundos**.

- **Filtro de Calidad:** Solo muestra coches con QualityScore > 80 (sin accidentes, fotos HD, vendedor verificado).
- **Visualización:**
  - **Asset:** BMW M2 CS
  - **Time:** 00:04s (Contador regresivo).
  - **Badge:** NEW BLOCK.

---

3. ARQUITECTURA "BARE METAL" (CÓMO SE LOGRA EL 13/10)

Para que esto no colapse el navegador con miles de datos, usamos técnicas de **HFT (High Frequency Trading)**.

A. BACKEND (GOLANG + REDIS PIPELINES)

- **Motor de Eventos: Go (Fiber)**. Gestiona la concurrencia.
- **Bus de Mensajes: Redis Pub/Sub**.
  - Los Scrapers (Python) inyectan eventos en canales Redis (scanner:arbitrage, scanner:panic).
  - Go escucha y retransmite.
- **Serialización: Protocol Buffers (Protobuf)** en lugar de JSON.
  - *Por qué:* JSON es texto pesado ("price": 20000). Protobuf es binario puro (0x4E20). Reduce el payload un 60%.
  - *Resultado:* Latencia de red insignificante incluso en conexiones móviles 4G.

B. FRONTEND (REACT VIRTUALIZED + SVG)

- **Virtual Windowing:** Usamos react-window o tanstack-virtual.
  - *Por qué:* Si cargas 5.000 filas en el DOM (HTML), Chrome explota.
  - *Solución:* Solo renderizamos las 30 filas que ves en pantalla. Al hacer scroll, reciclamos los componentes DOM en microsegundos. Sensación de "Scroll Infinito" a 120 FPS.
- **Sparklines (Micro-Gráficos):**
  - No usamos librerías de charts pesadas aquí.
  - Generamos un **SVG Path String** (d="M0 10 L10 5 L20 15...") directamente en el servidor o vía WebAssembly. Cero impacto en JS Main Thread.

---

4. INTERACCIÓN "ONE-CLICK KILL" (UX DE COMBATE)

El Scanner no es informativo. Es ejecutivo.

Cada fila es un **Trigger Activo**.

MENÚ TÁCTICO (CLICK DERECHO / LONG PRESS):

1. **EXECUTE ORDER:** Lanza el bot de reserva automática (si la API del portal lo permite).
2. **LOCK SOURCE:** Bloquea el anuncio. (Copia la URL al portapapeles, abre WhatsApp Web con mensaje pre-redactado al vendedor: *"Hola, compro tu [Coche] ya. Pago contado."*).
3. **SYNTHETIC LIST:** (Si activamos el módulo Dropshipping) Clona el anuncio a tu web instantáneamente.
4. **DEEP SCAN:** Abre el gráfico TradingView principal para análisis técnico profundo.

---

5. ESTRUCTURA DE DATOS (EL PAYLOAD BINARIO)

Así viaja la información por el cable. Densidad máxima.

Protocol Buffers

```
message ScannerItem {
  string id = 1;            // ID Único Hash
  string ticker = 2;        // "BMW.M4.COMP"
  double price = 3;         // 84500.00
  double delta = 4;         // -2500.00 (Bajada)
  int32 timestamp = 5;      // Unix Epoch
  enum SignalType {
    ARBITRAGE = 0;
    PANIC = 1;
    VELOCITY = 2;
    FRESH = 3;
  }
  SignalType signal = 6;
  string meta_json = 7;     // Datos extra compactos (ROI, Margin)
```

```
  bytes sparkline = 8;        // Vector SVG comprimido
}
```

---

6. HOJA DE RUTA DE IMPLEMENTACIÓN ESTRUCTURAL

FASE 1: EL MOTOR (SEMANA 1)

- Configurar **Redis Streams** en el Hetzner.
- Escribir los 4 scripts en Python (Workers) que calculan las matemáticas de los canales A, B, C, D y publican en Redis.

FASE 2: EL TÚNEL (SEMANA 2)

- Desarrollar el servidor **Go WebSocket Gateway**.
- Implementar codificación **Protobuf**.

FASE 3: EL VISOR (SEMANA 3)

- Montar el componente VirtualList en Next.js.
- Diseñar las filas "Condensadas" (CSS Grid ultra-compacto).
- Integrar los colores de semáforo (Verde/Rojo/Naranja/Blanco).

FASE 4: EL GATILLO (SEMANA 4)

- Programar el menú contextual de ejecución rápida.
- Conectar con la API de WhatsApp/Email para ataque inmediato.

---

7. COSTES DIRECTOS (MÓDULO 6)

- **Infraestructura Backend:** 0€ (Usa el mismo Hetzner AX102 y Redis ya desplegado).
- **Ancho de Banda:** Marginal (Gracias a Protobuf y WebSockets).
- **Coste Desarrollo: Alta Complejidad**. Requiere un experto en Go y optimización Frontend.
    - Estimación: **6.000€ - 8.000€** (Freelance Top Tier / Agencia).

ESTO ES EL MÓDULO 6.

Un radar que funciona más rápido que el pensamiento humano.

Filtra la basura. Te da el oro. Te permite disparar en un clic.

THE SCANNER — HFT OPPORTUNITY RADAR (MASTER BLUEPRINT)

CLASIFICACIÓN: SISTEMA DE ARBITRAJE Y ALERTA TEMPRANA DE BAJA LATENCIA

ARQUITECTURA: EVENT-DRIVEN (REDIS PUB/SUB + WEBSOCKETS + PROTOBUF)

INFRAESTRUCTURA: DESPLIEGUE DUAL (VPS 1 + VPS 2 VÍA LAN PRIVADA)

LATENCIA OBJETIVO: < 10ms (INTERNAL) / < 50ms (CLIENT DELIVERY)

---

1. MISIÓN CRÍTICA (EL "QUÉ")

El Scanner no es un buscador pasivo. Es un **MOTOR DE EJECUCIÓN ACTIVA**.

Su función es procesar el flujo de datos brutos (Raw Data) en tiempo real, aplicar 4 filtros matemáticos de rentabilidad y "empujar" (Push) solo las oportunidades de oro a la pantalla del usuario antes de que el mercado reaccione.

---

2. ARQUITECTURA DE FLUJO DE DATOS (EL "CÓMO" GLOBAL)

El sistema funciona como una tubería de alta presión:

1. **ORIGEN (VPS 1 - EL CAZADOR):** Los Scrapers (Python) extraen datos. En lugar de guardar en DB y esperar, analizan *in-situ*.
2. **TRANSPORTE (REDIS PUB/SUB):** Si el dato pasa el filtro, se serializa (binario) y se dispara al canal Redis.
3. **DISTRIBUCIÓN (VPS 2 - EL ANALISTA):** Un servidor en **Go (Golang)** escucha Redis y retransmite vía **WebSockets**.
4. **VISUALIZACIÓN (CLIENTE):** React renderiza la oportunidad en el Sidebar.

---

3. LOS 4 MOTORES DE INTELIGENCIA (LÓGICA + CÓDIGO)

Aquí se fusiona el negocio con la implementación técnica.

MOTOR A: ARBITRAGE KINGS (DIFERENCIALES GEOGRÁFICOS)

- **Negocio:** Explota la ineficiencia de precio entre países (ej: Comprar en DE, Vender en ES).

- **Matemática:**
  $$Spread = (P_{Destino} \times (1 - IVA_{Dest})) - (P_{Origen} + Logística + Matriculación)$$
  $$ROI = \frac{Spread}{P_{Origen}} \times 100$$

Implementación Técnica (Python en VPS 1):
Python
```
# Lógica inyectada en el loop del Scraper

def check_arbitrage(car_obj):

    # Estimación en tiempo real del precio mercado destino (cacheado en RAM)

    target_price = market_cache.get(car_obj.model_id, default=0)



    costs = 1200 # Logística + Gestoría

    net_spread = (target_price / 1.21) - (car_obj.price + costs)



    if net_spread > 2500 and (net_spread / car_obj.price) > 0.12:

        # ÉXITO: Empaquetar y enviar

        payload = protobuf_pack(id=car_obj.id, type="ARB", val=net_spread)

        redis_client.publish('scanner_feed', payload)
```

-

MOTOR B: PANIC & DISTRESS (VOLATILIDAD NEGATIVA)

- **Negocio:** Detecta bajadas de precio violentas (>5% en <1h) que indican necesidad urgente de liquidez.
- **Matemática:**
  $$\Delta P = \frac{P_{t} - P_{t-1}}{P_{t-1}}$$

Implementación Técnica (Redis en VPS 2):
Usamos Redis como memoria de corto plazo para no machacar la base de datos principal.
Python
```
def check_panic(car_obj):

    # Redis Key: "price_history:CAR_ID"

    last_price = redis_client.get(f"ph:{car_obj.id}")
```

```
if last_price and car_obj.price < (float(last_price) * 0.95):

    # ÉXITO: Caída > 5% detectada

    drop_val = float(last_price) - car_obj.price

    payload = protobuf_pack(id=car_obj.id, type="PNC", val=drop_val)

    redis_client.publish('scanner_feed', payload)


# Actualizamos precio para la próxima

redis_client.setex(f"ph:{car_obj.id}", 86400, car_obj.price)
```

-

MOTOR C: VELOCITY VORTEX (ABSORCIÓN)

- **Negocio:** Identifica modelos que se venden en <48h. Señal de "Compra Ciega" por escasez.
- **Matemática:**
  $$Absorption = \frac{\text{Sales}_{24h}}{\text{Stock}_{Total}}$$

Implementación Técnica (ClickHouse + Python):
Este filtro es pre-calculado. El scraper consulta una *Bloom Filter* o una *Look-up Table* en memoria.
Python

```
# La tabla 'hot_models' se actualiza cada hora desde ClickHouse

if car_obj.model_id in hot_models_set:

    # Si entra un coche de un modelo "Caliente", alerta inmediata

    payload = protobuf_pack(id=car_obj.id, type="VEL", val=hot_models[model].dts)

    redis_client.publish('scanner_feed', payload)
```

-

MOTOR D: MEMPOOL SNIPER (LATENCIA CERO)

- **Negocio:** Ver el coche antes que la indexación web pública.
- **Tecnología:** No hay filtro matemático, solo temporal.
- **Implementación:** Conexión directa al flujo de ingestión. Si QualityScore > 80, pasa directo al socket sin procesar. Latencia: 0.0s.

---

4. EL NÚCLEO DE VELOCIDAD (EL SERVIDOR)

No usamos Apache ni Nginx para esto. Usamos un microservicio dedicado.

- **Lenguaje: Go (Golang)**.
- **Ubicación:** VPS 2 ("El Analista").
- **Librerías:** Fiber (HTTP/WS), Redigo (Redis), Protobuf (Serialización).
- **Lógica del Servidor (Blueprint):**
    1. Abre un pool de conexiones WebSocket (ws://scan.cardex.com).
    2. Se suscribe al canal Redis scanner_feed.
    3. Cuando llega un mensaje binario (Protobuf), hace un **Broadcast** a todos los clientes conectados.
    4. **Zero-Copy:** No decodifica el mensaje. Lo pasa tal cual (binario) del Redis al WebSocket para ahorrar CPU.

---

5. ESTRUCTURA DE DATOS (EL PAYLOAD)

Para máxima velocidad, los datos viajan en binario (Protocol Buffers), no en texto (JSON).

Definición .proto:

Protocol Buffers

```
message ScannerAlert {

  string id = 1;          // ID del anuncio

  string ticker = 2;      // "BMW.M4.G82"

  double price = 3;       // Precio actual

  double metric = 4;      // Valor del Spread o Drop

  int32 signal_type = 5;  // 1=Arb, 2=Panic, 3=Vel, 4=Fresh

  string url = 6;         // Link directo al origen

  int64 timestamp = 7;    // Momento exacto de detección

}
```

- **Tamaño del paquete:** ~60 bytes. (JSON equivalente: ~250 bytes).
- **Ventaja:** Permite enviar miles de alertas por segundo incluso en conexiones móviles 4G.

---

6. FRONTEND: LA INTERFAZ DE COMBATE

El usuario recibe estos bytes y el navegador los pinta.

- **Tecnología: React** + **react-window** (Virtualización).
- **Componente ScannerSidebar:**
  - Mantiene un Buffer circular de 100 elementos en memoria.
  - Solo renderiza los 20 visibles en el DOM.
- **Estética Institucional (CSS):**
  - Fondo: #000000 (OLED Black).
  - Fuente: JetBrains Mono o Roboto Mono (11px).
  - **Código de Colores Semánticos:**
    - **ARB (Arbitraje):** Verde Matrix (#00FF41).
    - **PNC (Pánico):** Rojo Sangre (#FF1A1A) + Parpadeo CSS animation: flash 1s.
    - **VEL (Velocidad):** Naranja Ámbar (#FFAA00).
    - **NEW (Nuevo):** Blanco Puro (#FFFFFF).

---

7. INTERACCIÓN TÁCTICA (EXECUTION)

El Scanner no sirve si no puedes actuar.

1. **Click Izquierdo:** Carga el activo en el **Gráfico Principal (TradingView)** para análisis técnico.
2. **Click Derecho (Menú Contextual):**
   - **[LOCK]:** Copia URL + Mensaje WhatsApp predefinido ("Te lo compro ya").
   - **[CLONE]:** (Si Módulo Dropshipping activo) Publica el coche en tu web instantáneamente.
   - **[IGNORE]:** Elimina la fila y entrena al algoritmo (Feedback loop).

---

8. DESPLIEGUE "BOOTSTRAP" (PASO A PASO)

Cómo montas esto HOY con 0€ en tus 2 VPS:

1. **Configuración de Red:**
   - Activa la **Hetzner vSwitch LAN** entre VPS 1 y VPS 2.
   - Asegura que Redis en VPS 2 escucha en la IP interna (bind 10.0.0.2).
2. **Despliegue Backend (VPS 2):**
   - Instala Go.
   - Crea el archivo server.go (Generado por IA con las especificaciones de arriba).
   - Ejecuta: go run server.go.
3. **Inyección (VPS 1):**
   - Modifica tu script Python actual. Añade la librería redis.
   - Pega las funciones lógicas (check_arbitrage, check_panic) dentro del bucle de extracción.
4. **Frontend (Cliente):**

- Añade el componente React a tu proyecto Next.js.
- Conecta al WebSocket.

CARDEX_SYSTEM_KERNEL.md

1. DATA SCHEMA (THE TRUTH)

Engine: ClickHouse (VPS 2)

Optimization: MergeTree (Write-Heavy)

SQL

```
-- TABLA MAESTRA DE ACTIVOS (Coches como Acciones)

CREATE TABLE cardex_core.market_assets (

    timestamp DateTime64(3) CODEC(Delta, ZSTD), -- Precisión ms

    ticker_id LowCardinality(String),          -- Ej: "BMW.M4.G82.COMP"

    vin String,                                -- Unique ID

    price Float32,                             -- Precio Listado

    currency LowCardinality(String) DEFAULT 'EUR',


    -- METRICAS DE GRADO MILITAR

    alpha_score Float32,                       -- Calculado por Oracle (0-100)

    absorption_rate Float32,                   -- Velocidad de venta (0-1)

    spread_arb Float32,                        -- Margen vs Mercado Objetivo


    -- FUENTE Y RASTREO

    seller_id String,                          -- Hash del Vendedor

    source_node LowCardinality(String),        -- "DE_MOBILE", "IT_AUTOSCOUT"

    is_smart_money UInt8                       -- 1 = Top 10 Dealer (Oracle)

)

ENGINE = MergeTree()
```

```
PARTITION BY toYYYYMM(timestamp)
ORDER BY (ticker_id, timestamp, alpha_score)
TTL timestamp + INTERVAL 5 YEAR;

-- VISTA MATERIALIZADA PARA TRADINGVIEW (Velas OHLC en tiempo real)
CREATE MATERIALIZED VIEW cardex_core.ohlc_1h
ENGINE = AggregatingMergeTree() ORDER BY (ticker_id, time_bucket)
AS SELECT
    toStartOfHour(timestamp) as time_bucket,
    ticker_id,
    argMin(price, timestamp) as open,
    max(price) as high,
    min(price) as low,
    argMax(price, timestamp) as close,
    uniq(vin) as volume_units,
    avg(absorption_rate) as liquidity_index
FROM cardex_core.market_assets
GROUP BY ticker_id, time_bucket;
```

---

2. COMMUNICATION PROTOCOL (THE NERVES)

System: Redis Pub/Sub + Protobuf

Stream: scanner_feed (Latencia < 5ms)

PROTOBUF DEFINITION (scanner.proto):

Protocol Buffers

```
syntax = "proto3";
```

```
package cardex.hft;

message MarketSignal {
  string asset_ticker = 1;      // "PORSCHE.911.GT3"
  double strike_price = 2;      // 185000.00
  double arbitrage_delta = 3;   // +12500.00 (Ganancia Neta)

  enum SignalType {
    ARBITRAGE_GAP = 0;          // Diferencial Geográfico
    LIQUIDITY_VORTEX = 1;       // Escasez Extrema
    PANIC_DUMP = 2;             // Caída > 5% en 1h
    WHALE_ENTRY = 3;            // Smart Money (Oracle) comprando
  }
  SignalType type = 4;

  int64 timestamp_us = 5;       // Unix Microseconds
  string source_url_hash = 6;   // Link encriptado para ejecución
}
```

---

3. INFRASTRUCTURE TOPOLOGY (THE MAP)

Network: Hetzner vSwitch (Private LAN 10G)

NODE	ROLE	INTERNAL IP	SERVICES RUNNING

VPS 1 (HUNTER)	**Ingestion & Math**	10.0.0.2	• Python Scrapers (x50 Workers) • Oracle Bots (x10 Snipers) • Redis Publisher (Python)
VPS 2 (ANALYST)	**Storage & Gateway**	10.0.0.3	• ClickHouse Server (Port 8123) • Redis Broker (Port 6379) • **Go WebSocket Gateway** (Port 8080) • Next.js SSR (Port 3000)

---

4. EXECUTION LOGIC (THE CODE)

Component: ScannerGateway (Golang)

Location: VPS 2 (/opt/cardex/gateway)

Go

```
// EL NUCLEO QUE MUEVE EL MODULO 6 Y 11

func main() {
```

```
    // 1. Conexión a Redis (Memoria Compartida)
    rdb := redis.NewClient(&redis.Options{Addr: "10.0.0.3:6379"})
    pubsub := rdb.Subscribe(ctx, "scanner_feed")

    // 2. WebSocket Hub (Gestión de 10k conexiones)
    m := melody.New()

    // 3. Loop de Alta Frecuencia (Zero-Copy Broadcast)
    go func() {
        for msg := range pubsub.Channel() {
            // Recibe binario Protobuf -> Empuja directo al Socket
            // NO DECODIFICA. VELOCIDAD PURA.
            m.BroadcastBinary([]byte(msg.Payload))
        }
    }()

    // 4. Endpoint Seguro
    http.HandleFunc("/ws/feed", func(w http.ResponseWriter, r *http.Request) {
        m.HandleRequest(w, r) // Upgrade HTTP -> WSS
    })

    http.ListenAndServe(":8080", nil)
}
```

---

5. CAPABILITIES MATRIX (FINAL AUDIT)

MODULE	TECH STACK	LATENCY	STATUS
1. INGESTION	Python + Antigravity (Nodriver)	N/A	**READY**
2. DATABASE	ClickHouse (MergeTree)	4ms (Query)	**DEFINED**
3. TRADINGVIEW	Lightweight Charts + OHLC View	60fps	**READY**
5. ORACLE	LangGraph + AlphaScore Algorithm	60s (Update)	**ARCHITECTED**
6. SCANNER	Go + Redis + Protobuf	<50ms (Push)	**ARCHITECTED**
11. RADAR	React Virtualized + SVG	120fps	**READY**